

以 H5ST 部分为例的详细的小白级别的初级的 JSVMP 插桩纯算入门案例

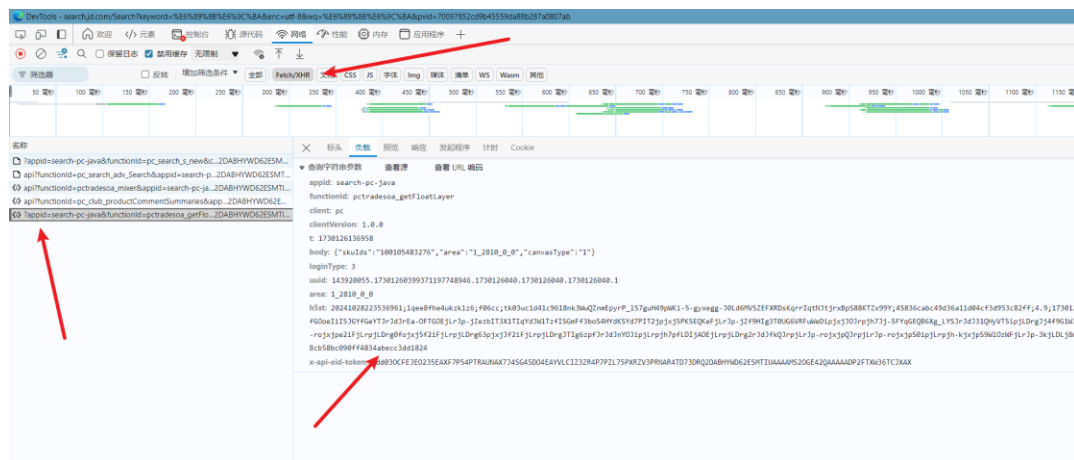
---兼讲一下 Base64 编码是怎么回事

第一章：先初步分析

基本的跟栈和逆向以及控制台的操作就不讲了。

打开 JD 页面，搜索一个东西，例如“手机”。

会在得到网络请求响应。在请求负载里可以看到一个 H5ST 的参数。



该参数用“;”分开，一共有 9 段。目前 H5ST 版本 4.9.1

根据网上大佬们的分析，各部分的含义如下：

第一部分，时间戳转化的时间字符串

第二部分，fp 值，由一个初始的种子字符串，不断计算得出的 16 位字符串，末尾永远是数字

第三部分，appid 值，接口不同值不同

第四部分，tk 值，接口返回，tk 和随机算法绑定，过期失效

第五部分，此部分值为 MD5 加密结果，但是再进行 md5 之前会有一系列操作，过程相对复杂

第六部分，版本号

第七部分，时间戳，与第一部分相关

第八部分，环境等参数经过魔改 base64 编码后的结果

第九部分，MD5 值，跟第五部分生成方式一致，过程字符串不一样

这里我们着重以第八部分为例来讲解 JSVMP 如何分析和插桩。

在控制台里搜索 H5ST 就能查找到 H5ST 的入口位置。经过调试发现 search_new.init.js 的第 178 行为入口。

```
Q H5ST

▼ search_new.init.js?2024-10-25-01! — search.jd.com/script/search_new.init.js?2024-10-25-01!
94 var h5stURI = encodeURI(signedParams.h5st)
94 ...= encodeURI(signedParams.h5st)
95 colorParam['h5st'] = h5stURI;
95 colorParam['h5st'] = h5stURI;
178 var h5stURI = encodeURI(signedParams.h5st)
178 ...= encodeURI(signedParams.h5st)
179 colorParam['h5st'] = h5stURI;
179 colorParam['h5st'] = h5stURI;

▼ colorApi.js — storage.360buyimg.com/retail-mall/search_list/pc/product/search/0.0.17/js/colorApi.js
2 ...e).then(function(a){g+="&h5st="+encodeURI(a.h5st);try{getJsToken(function(a){a&&a.jsToken&&(g+="&x-api-eid-token="+a.jsToken),c(g,d)),600))
2 ...[g+="&h5st="+encodeURI(a.h5st);try{getJsToken(function(a){a&&a.jsToken&&(g+="&x-api-eid-token="+a.jsToken),c(g,d)),600)}catch(b){})}catch(f

▼ colorApi.js — storage.360buyimg.com/retail-mall/search_list/pc/product/search/0.0.26/js/colorApi.js
2 ...g).then(function(b){i+="&h5st="+encodeURI(b.h5st),f(i,e,c,a))["catch"](function(b){f(i,e,c,a),d(JSON.stringify({data:b}),a,param,"\u52a0\u7b7e\u5f02\
2 ...[i+="&h5st="+encodeURI(b.h5st),f(i,e,c,a))["catch"](function(b){f(i,e,c,a),d(JSON.stringify({data:b}),a,param,"\u52a0\u7b7e\u5f02\u5e38",752))}catcl

▼ js-security-v3-rac.js?v=20241028 — storage.360buyimg.com/webcontainer/main/js-security-v3-rac.js?v=20241028
3 ...r Lx=px.exports;var Nx=["h5st","_stk","_ste"];function zx(t,e){e=e||0;for(var r=t.length-e,n=new Array(r);r--;)n[r]=t[r+e];return n}function Bx(t){return r
3 ...return(0,w.assign)(t,{s:e:h5st?encodeURI(e.h5st):null}})).catch((function(){return(0,w.assign)(t,{s:null}})))))catch(t){return function(t){return new d.defaul
3 ...((t,{s:e:h5st?encodeURI(e.h5st):null}})).catch((function(){return(0,w.assign)(t,{s:null}})))))catch(t){return function(t){return new d.default((function(e){e((

▼ Search?keyword=%E6%89%8B%E6%9C%BA&enc=utf-8&wq=%E6%89%8B%E6%9C%BA&pvid=70097852cd9b45559da88b287a08... — search.jd.com,
8471 ..."text/javascript" src="//h5static.m.jd.com/act/jd-jssdk/latest/jd-jssdk.min.js"></script>
```

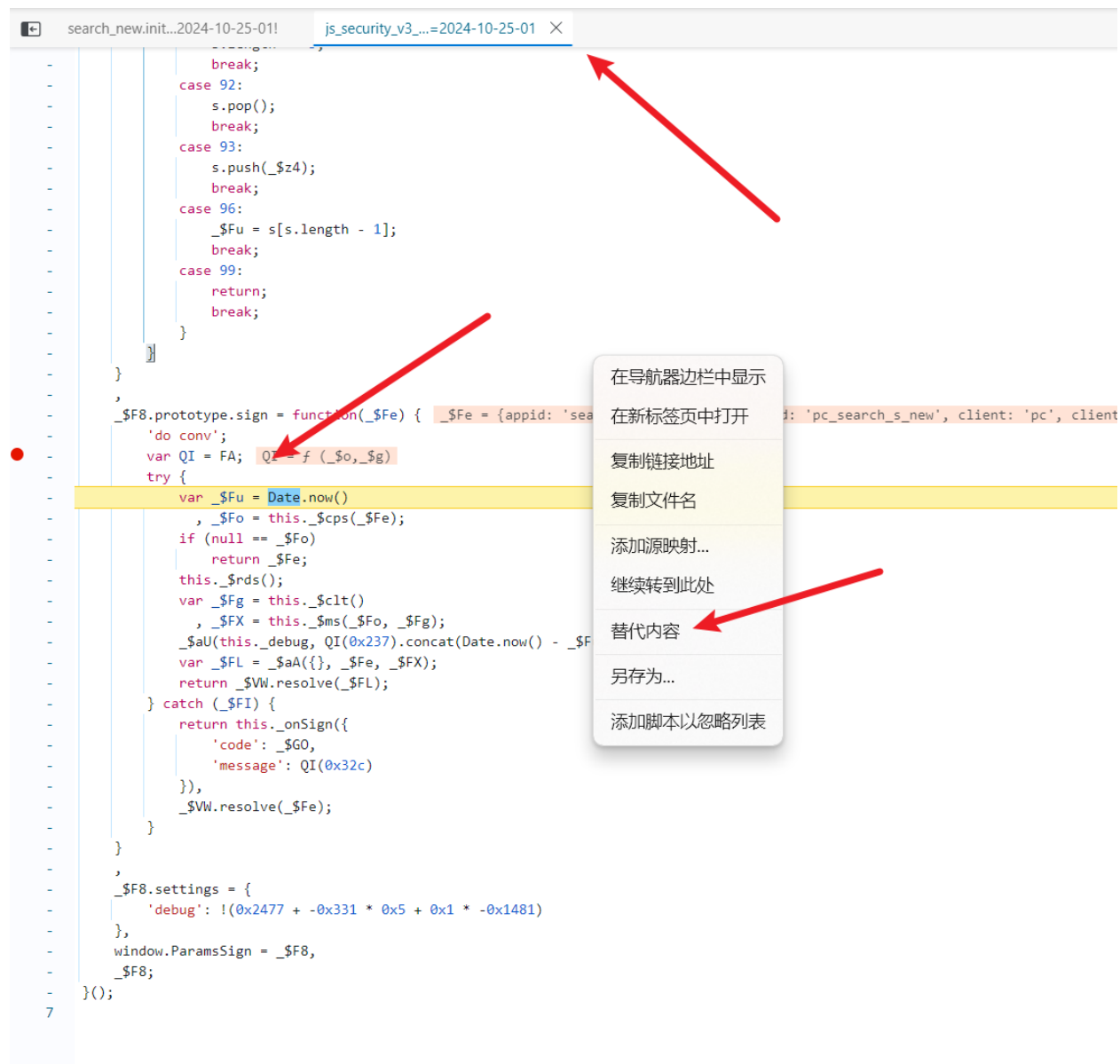
打上断点跳入进去。注意断点要打在 `sign` 前面，不要打在 `window.PSign` 前面。

```

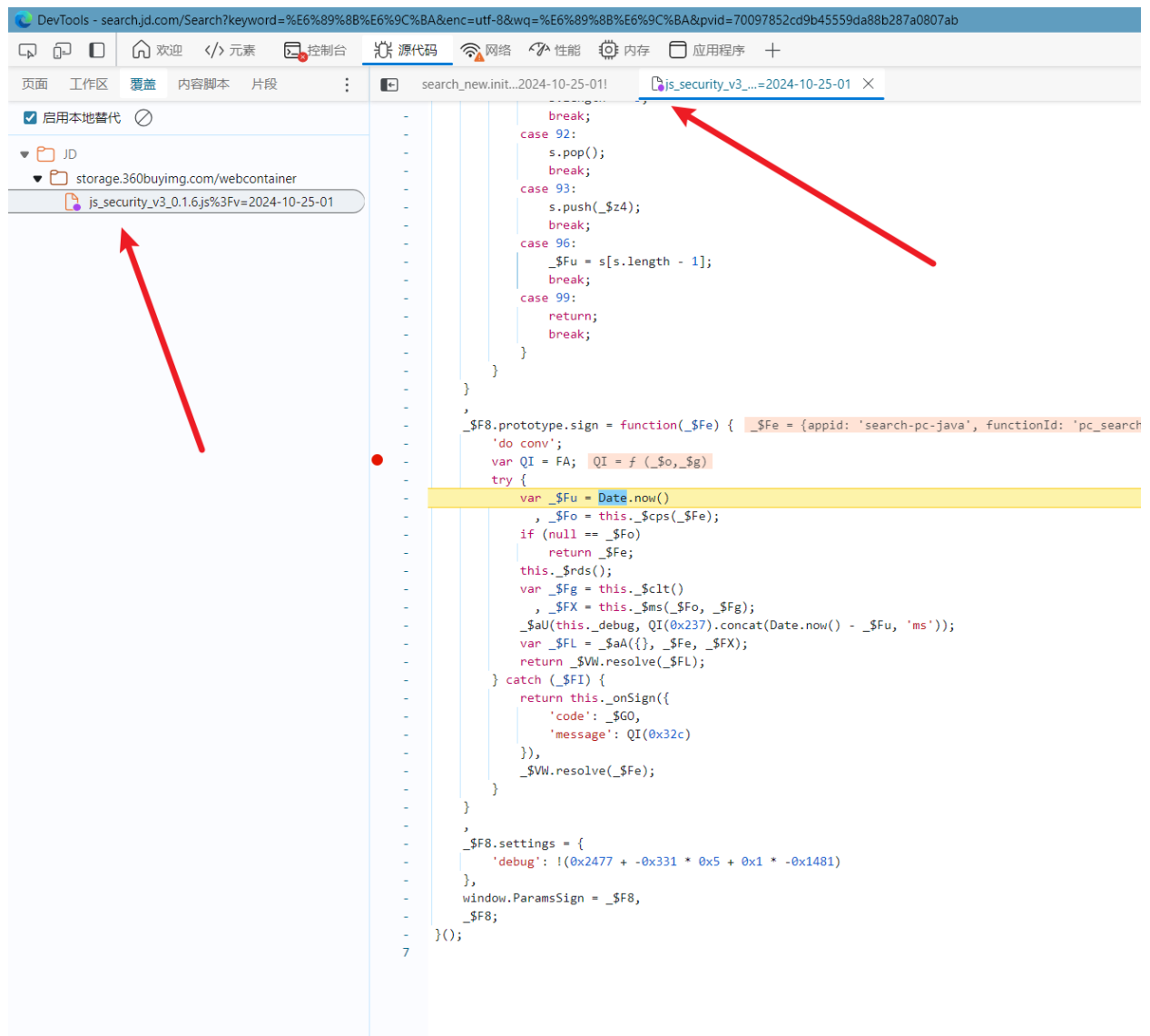
search_new.init...2024-10-25-01! X
158 var jda = readCookie('__jda');
159 var djd = readCookie('ipLoc-djd');
160 var area = (djd!=null && djd.length>0)?djd.split('.')[0].replace(new RegExp("-", 'g'), '_'):"";
161 var colorParam = {
162     appid: 'search-pc-java',
163     functionId: functionId,
164     client: 'pc',
165     clientVersion: '1.0.0',
166     t: new Date().getTime(),
167     body: JSON.stringify(paramJson)
168 }
169
170 try{
171     var colorParamSign = JSON.parse(JSON.stringify(colorParam))
172     colorParamSign['body'] = SHA256(colorParam.body);
173     colorParam['loginType'] = '3';
174     colorParam['uuid'] = jda;
175     colorParam['area'] = area;
176     var that = this;
177     window.PSign.sign(colorParamSign).then(function(signedParams){
178         var h5stURI = encodeURI(signedParams.h5st)
179         colorParam['h5st'] = h5stURI;
180         //var url = getColorApiUrl(functionId,paramJson);
181         getJsToken(function(res){
182             if(res && res.jsToken){
183                 colorParam['x-api-eid-token'] = res.jsToken;
184             }
185             $.ajax({
186                 url: colorApi,
187                 async: true,
188                 data : colorParam || {},
189                 //dataType: 'json',
190                 xhrFields: {
191                     withCredentials: true,
192                 },
193                 success: function(data) {
194                     callback(data);
195                     if(data && data.startsWith("{") ){
196                         var resObj = JSON.parse(data);
197                     }
198                 }
199             });
200         });
201     });
202 }

```

会跳转到 js_security_v3 这个 js 文件中，var QI=FA 这里是入口。右键点击替代内容，这样可以在本地映射一个副本，便于调试。



这里有一个紫色的点点就是替代好了。



我们继续忽略跟栈调试过程，直接进到 `clt` 这个方法中，这个方法就是第八段自定义 Base64 编码的入口。

```
search_new.init...2024-10-25-01!  js_security_v3_...=2024-10-25-01 X
-         break;
-     case 92:
-         s.pop();
-         break;
-     case 93:
-         s.push(_$z4);
-         break;
-     case 96:
-         _$Fu = s[s.length - 1];
-         break;
-     case 99:
-         return;
-         break;
-     }
- }
- }
-
- _$F8.prototype.sign = function(_$Fe) {  _$Fe = {appid: 'search-pc-java', functionId: 'pc_search_s
-     'do conv';
-     var QI = FA;  QI = f (_$o, _$g)
-     try {
-         var _$Fu = Date.now()  _$Fu = 1730126852401
-         , _$Fo = this._$cps(_$Fe);  _$Fo = (6) [{...}, {...}, {...}, {...}, {...}, {...}], _$Fe = {appid: '
-         if (null == _$Fo)
-             return _$Fe;  _$Fe = {appid: 'search-pc-java', functionId: 'pc_search_s_new', client:
-         this._$rds();
-         var _$Fg = this._$clt()
-         , _$FX = this._$ms(_$Fo, _$Fg);
-         _$aU(this._debug, QI(0x237).concat(Date.now() - _$Fu, 'ms'));
-         var _$FL = _$aA({}, _$Fe, _$FX);
-         return _$VW.resolve(_$FL);
-     } catch (_$FI) {
-         return this._onSign({
-             'code': _$GO,
-             'message': QI(0x32c)
-         }),
-         _$VW.resolve(_$Fe);
-     }
- }
-
- _$F8.settings = {
-     'debug': !(0x2477 + -0x331 * 0x5 + 0x1 * -0x1481)
- },
- window.ParamsSign = _$F8,
- _$F8;
- }();
7
```

进到 `_$F8.prototype._$clt` 这个函数中发现是一大堆控制流和 JSVMP。JSVMP 其实就是定制化的一个堆栈，不断的通过向堆栈内压入函数或变量，（用 `push` 指令），或者取出函数或变量（用 `pop` 指令），或者运行某个函数（一般用 `call` 或 `apply` 来运行）。

通常的做法是向堆栈中压入一个函数和几个变量。再用 `call` 或 `apply` 来运行这个堆栈中的函数和变量。运行完毕后再把结果保存在堆栈中，或者另外赋值出去，再用 `pop` 指令把没用的函数和变量剔除出去。然后再重复以上步骤进行下一个计算。

```
-     }
-     ,
-     _$F8.prototype._$clt = function() {
-         'use strict';
-         var h = 3r1k1;
-         var j = _2dpk1;
-         var QL, _$Fe, _$Fu;
-         var s = [];
-         var e = 3698;
-         var x, r;
-         133: for (; ; ) {
-             switch (j[e++]) {
-                 case 9:
-                     s.push(FA);
-                     break;
-                 case 10:
-                     s.push(_$Fe);
-                     break;
-                 case 12:
-                     x = s.pop();
-                     s[s.length - 1] += x;
-                     break;
-                 case 16:
-                     s[s.length - 5] = h.call(s[s.length - 5], s[s.length - 4], s[s.length - 3], s[s.length - 2], s[s.length - 1]);
-                     s.length -= 4;
-                     break;
-                 case 18:
-                     s.push(j[e++]);
-                     break;
-                 case 29:
-                     s.push(_$F7);
-                     break;
-                 case 30:
-                     s.push(null);
-                     break;
-                 case 31:
-                     _$Fe = s[s.length - 1];
-                     break;
-                 case 32:
-                     s[s.length - 1] = s[s.length - 1][_111k1[227 + j[e++]]];
-                     break;
-                 case 34:
-                     x = s.pop();
-                     s[s.length - 1] = s[s.length - 1] === x;
-                     break;
-                 case 35:
-                     s.push(QL);
-                     break;
-                 case 38:
-                     QL = s[s.length - 1];
-                     break;
```

这个时候我们可以观察一下，发现 `switch` 里的变量是 `j[e++]`，这个变量其实不用太去关心它，它就是一个控制流用的参数而已。我们重点要观察的是在这个控制流中有哪些变量有被 `push` 和 `pop` 的操作。我们观察一下，可以看到主要是 `s` 这个变量，就是目前 JSVMP 运行的堆栈，有大量的 `push` 和 `pop` 操作。这个 `s` 就是我们要重点关注的对象。但是这里有很多 `case`，我们应当在哪里插桩呢？一般情况先只插 `call` 和 `apply` 即可。

这里可以用日志断点的方式插桩，也可以直接在 `js` 文件中插入 `console.log()` 来插桩。（注意，有时候 JSVMP 运行时读取 `log` 作为结果，所以如果插入 `console.log()` 后发现输出结果变了，就要考虑改用日志断点来插桩了。这里为了方便演示，我们用 `console.log()` 来插桩。）

插桩的时候，建议做一些标识，这样的话就更容易的知道是具体哪里操作。Case16 这里因为有 5 个 arguments，太多了，我这里就先不打印 arguments 了，只打印一个结果即可。

```
8682     }
8683   },
8684   _$f8.prototype._$slt = function() {
8685     'use strict';
8686     var h = _3rlk1;
8687     var j = _2dpk1;
8688     var QL, _$Fe, _$Fu;
8689     var s = [];
8690     var e = 3698;
8691     var x, r;
8692     133: for (; ; ) {
8693       switch (j[e++]) {
8694         case 9:
8695           s.push(FA);
8696           break;
8697         case 10:
8698           s.push(_$Fe);
8699           break;
8700         case 12:
8701           x = s.pop();
8702           s[s.length - 1] += x;
8703           break;
8704         case 16:
8705           console.log("=====case16=====");
8706           console.log("Arguments--->",h.call(s[s.length - 5], s[s.length - 4], s[s.length - 3], s[s.length - 2], s[s.length - 1]))
8707           s[s.length - 5] = h.call(s[s.length - 5], s[s.length - 4], s[s.length - 3], s[s.length - 2], s[s.length - 1]);
8708           s.length -= 4;
8709           break;
8710         case 18:
8711           s.push(j[e++]);
8712           break;
8713         case 29:
```

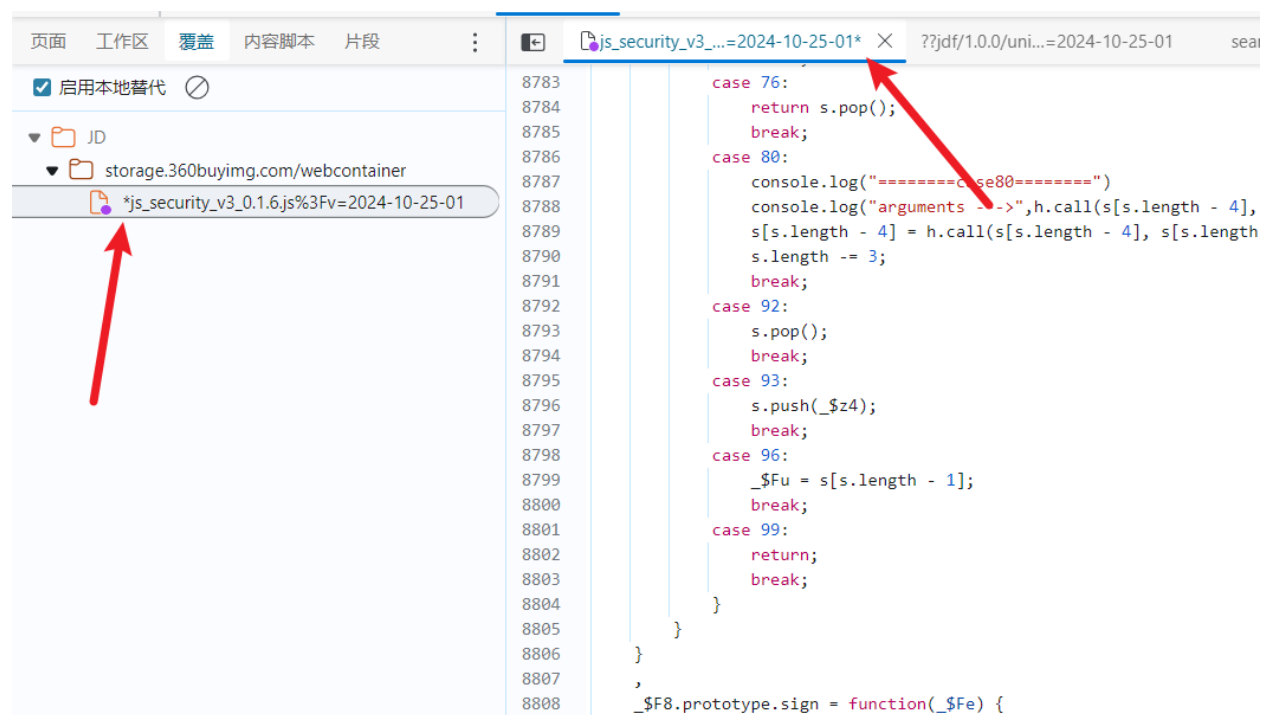
Case 47 这里有两种情况，要分别插不同的 log。这里可以大概猜一下这个 else 情况下在干什么：在 s 这个堆栈中倒数第 3 个应当是一个函数。这个函数赋值给了临时变量 x，然后以倒数第 1 个变量为参数去运行。运算的结果再覆盖了倒数第 3 个。最后删除掉倒数第 2 个和倒数第 1 个变量。

Case 80 我们也插入一个日志。

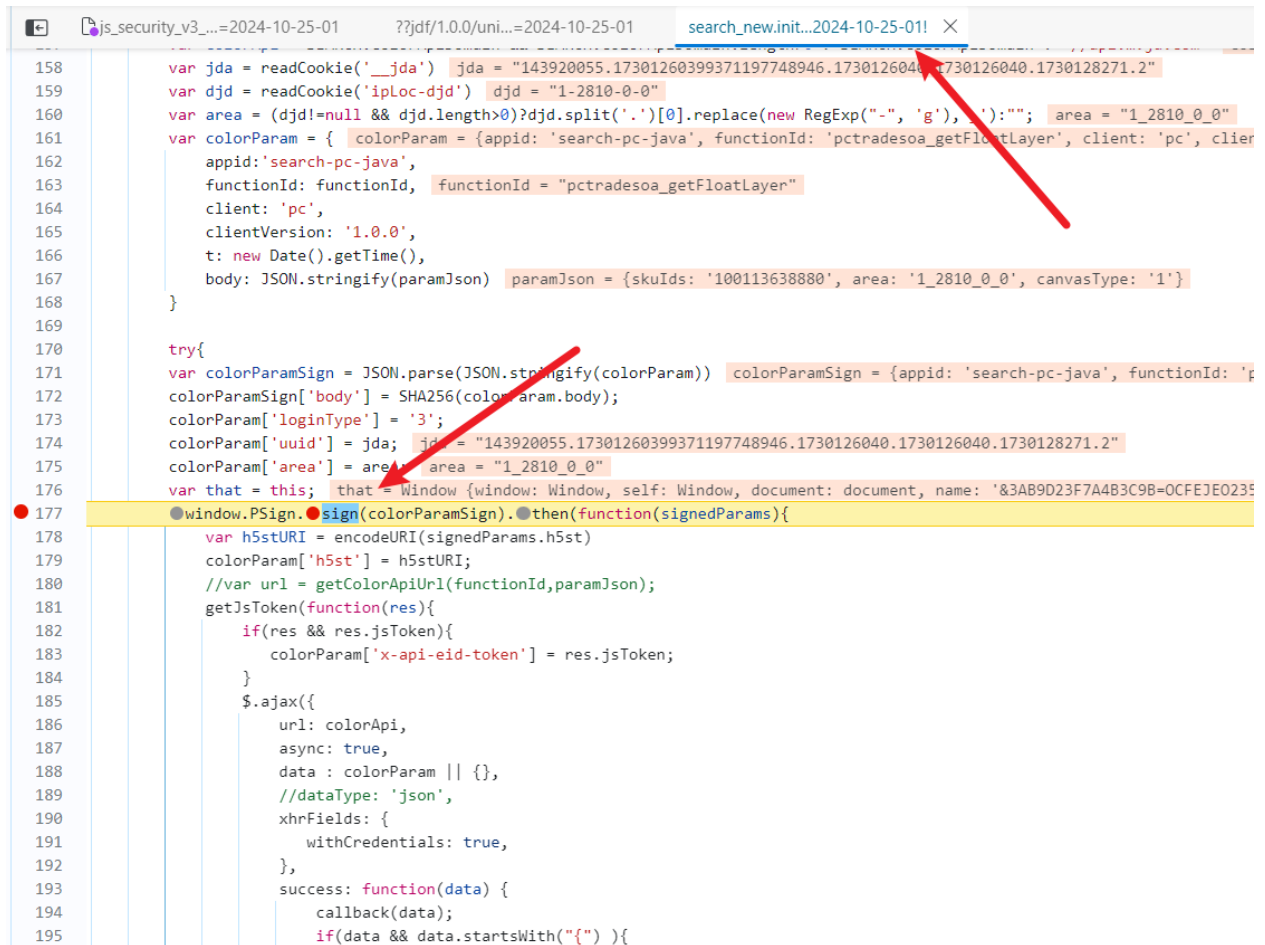
```
case 47:
  console.log("=====case47=====")
  if (s[s.length - 2] != null) {
    console.log("s[s.length - 2] --->",s[s.length - 2])
    console.log("arguments --->",h.call(s[s.length - 3], s[s.length - 2], s[s.length - 1]))
    s[s.length - 3] = h.call(s[s.length - 3], s[s.length - 2], s[s.length - 1]);
    s.length -= 2;
  } else {
    x = s[s.length - 3];
    console.log("s[s.length - 3] --->",s[s.length - 3])
    console.log("arguments --->", x(s[s.length - 1]))
    s[s.length - 3] = x(s[s.length - 1]);
    s.length -= 2;
  }
  break;

break,
case 80:
  console.log("=====case80=====")
  console.log("arguments --->",h.call(s[s.length - 4], s[s.length - 3], s[s.length - 2], s[s.length - 1]))
  s[s.length - 4] = h.call(s[s.length - 4], s[s.length - 3], s[s.length - 2], s[s.length - 1]);
  s.length -= 3;
  break;
```


最后别忘了 `controls + s` 保存一下文件。如果文件这里有星号，就是没有保存。没保存的文件是不生效的。

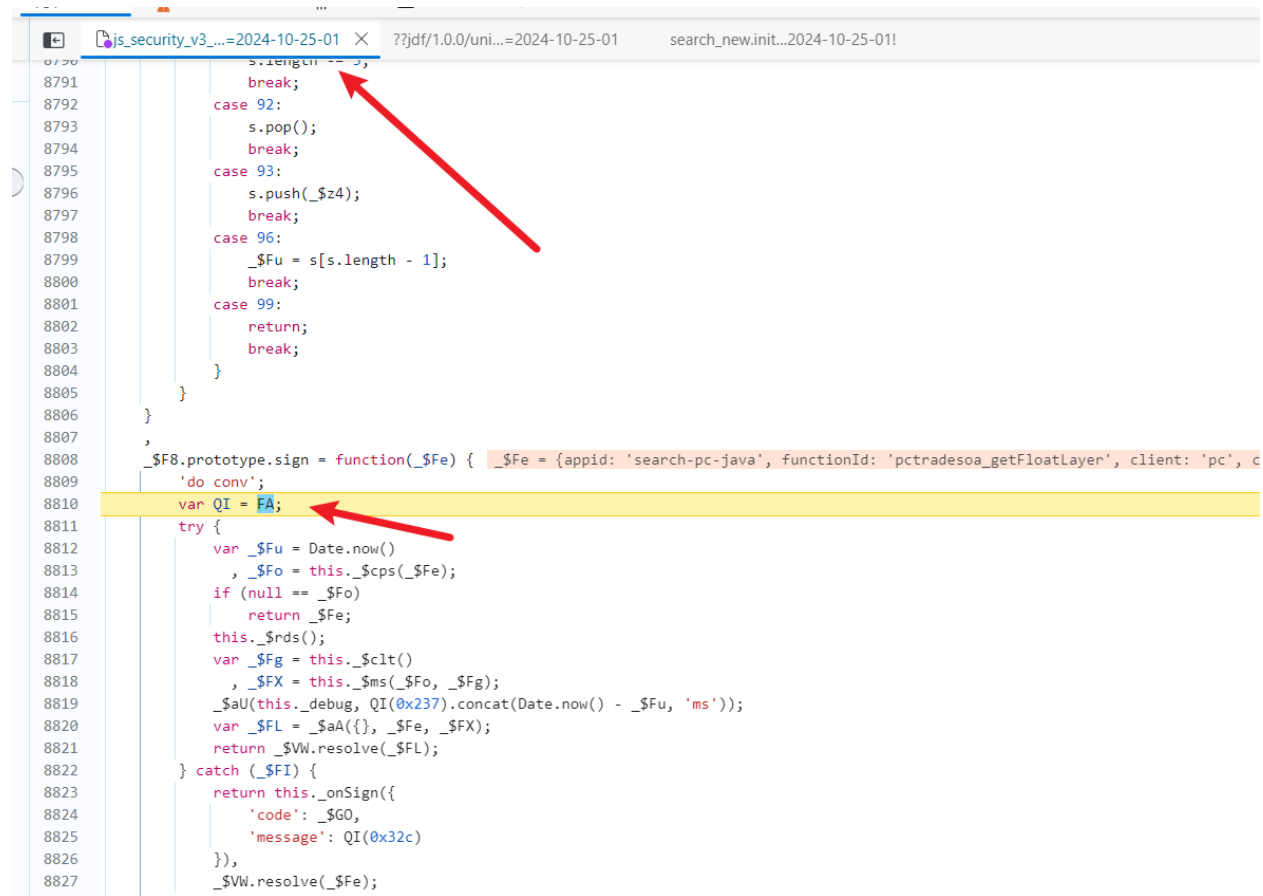


保存完了以后刷新页面，会断在我们打好断点的这里。还是一样跳进去。



```
158     var jda = readCookie('__jda')    jda = "143920055.17301260399371197748946.1730126040.1730126040.1730128271.2"
159     var djid = readCookie('iploc-djid')    djid = "1-2810-0-0"
160     var area = (djid!=null && djid.length>0)?djid.split('.')[0].replace(new RegExp("-", 'g'), '.'):"";    area = "1_2810_0_0"
161     var colorParam = {    colorParam = {appid: 'search-pc-java', functionId: 'pctradesoa_getFloatLayer', client: 'pc', clie
162         appid: 'search-pc-java',
163         functionId: functionId,    functionId = "pctradesoa_getFloatLayer"
164         client: 'pc',
165         clientVersion: '1.0.0',
166         t: new Date().getTime(),
167         body: JSON.stringify(paramJson)    paramJson = {skuIds: '100113638880', area: '1_2810_0_0', canvasType: '1'}
168     }
169
170     try{
171     var colorParamSign = JSON.parse(JSON.stringify(colorParam))    colorParamSign = {appid: 'search-pc-java', functionId: 'p
172     colorParamSign['body'] = SHA256(colorParam.body);
173     colorParam['loginType'] = '3';
174     colorParam['uuid'] = jda;    jda = "143920055.17301260399371197748946.1730126040.1730126040.1730128271.2"
175     colorParam['area'] = area;    area = "1_2810_0_0"
176     var that = this;    that = Window {window: Window, self: Window, document: document, name: '&3AB9D23F7A4B3C9B=OCFE3E0235
177     ●window.PSign.●sign(colorParamSign).●then(function(signedParams){
178     var h5stURI = encodeURI(signedParams.h5st)
179     colorParam['h5st'] = h5stURI;
180     //var url = getColorApiUrl(functionId,paramJson);
181     getJsToken(function(res){
182         if(res && res.jsToken){
183             colorParam['x-api-eid-token'] = res.jsToken;
184         }
185     }).ajax({
186         url: colorApi,
187         async: true,
188         data : colorParam || {},
189         //dataType: 'json',
190         xhrFields: {
191             withCredentials: true,
192         },
193         success: function(data) {
194             callback(data);
195             if(data && data.startsWith("{") ){
```

这时候就会发现断在了有我们紫色点点的文件的 var QI = FA 位置。这是因为我们替换了文件，这个打好日志的文件替代了原来的文件。



```
8790     s.length -= 1;
8791     break;
8792     case 92:
8793         s.pop();
8794         break;
8795     case 93:
8796         s.push(_$z4);
8797         break;
8798     case 96:
8799         _$Fu = s[s.length - 1];
8800         break;
8801     case 99:
8802         return;
8803         break;
8804     }
8805 }
8806
8807 ,
8808 _$F8.prototype.sign = function(_$Fe) { _$Fe = {appid: 'search-pc-java', functionId: 'pctradesoa_getFloatLayer', client: 'pc', c
8809     'do conv';
8810     var QI = FA;
8811     try {
8812         var _$Fu = Date.now()
8813             , _$Fo = this._$cps(_$Fe);
8814         if (null == _$Fo)
8815             return _$Fe;
8816         this._$rds();
8817         var _$Fg = this._$clt()
8818             , _$FX = this._$ms(_$Fo, _$Fg);
8819         _$aU(this._debug, QI(0x237).concat(Date.now() - _$Fu, 'ms'));
8820         var _$FL = _$aA({}, _$Fe, _$FX);
8821         return _$VW.resolve(_$FL);
8822     } catch (_$FI) {
8823         return this._onSign({
8824             'code': _$G0,
8825             'message': QI(0x32c)
8826         });
8827         _$VW.resolve(_$Fe);
8828     }
```

直接点击全部运行让所有的程序跑完。会在控制台生成很多日志。

我们翻到最上面一个一个的看。

最开始就运行的是 Case 47，就像上面说的，这里可以大概猜一下这个 else 在干什么：在 s 这个堆栈中倒数第 3 个应当是一个函数。这个函数是 function _\$F7(_\$Fe)。最后生成的结果是一个对象。

```
=====case47=====
s[s.length - 3] --> f _$F7(_$Fe) {
  'do conv';
  var LT = FA;
  var _$Fu = {};
  , _$Fo = ['pp', LT(0x297), LT(0x2c9), 'v', LT(0x2b8), 'pf', LT(0x305), LT(0x28a), LT(0x2da)];
  function _$F...
arguments --> {sua: 'Windows NT 10.0; Win64; x64', pp: {...}, extend: {...}, pf: 'Win32', random: 'pNrOoGGiDPNT', ...}
  canvas: "5b7096a993d442aef38ceb90cce4b485"
  ccn: 8
  extend: {wd: 0, l: 0, ls: 5, wk: 0, bu1: '0.1.6', ...}
  pf: "Win32"
  pp: {p2: 'dogcao'}
  random: "pNrOoGGiDPNT"
  sua: "Windows NT 10.0; Win64; x64"
  v: "h5_file_v4.9.1"
  webglFp: "a3e19bf2c47e1ad07028cb74d487f910"
  [[Prototype]]: Object
```

再往下看，case 16，看起来是把上面的对象给字符串化了。这个很可能就是我们要找的输入参数，可以先存起来。

```
=====case16=====
arguments--> {
  "sua": "Windows NT 10.0; Win64; x64",
  "pp": {
    "p2": "dogcao"
  },
  "extend": {
    "wd": 0,
    "l": 0,
    "ls": 5,
    "wk": 0,
    "bu1": "0.1.6",
    "bu2": -1,
    "bu3": 49,
    "bu4": 0,
    "bu5": 0,
    "bu6": 30,
    "bu7": "",
    "bu8": 0
  },
  "pf": "Win32",
  "random": "AzlVETij69L_",
  "v": "h5_file_v4.9.1",
  "canvas": "5b7096a993d442aef38ceb90cce4b485",
  "webglFp": "a3e19bf2c47e1ad07028cb74d487f910",
  "ccn": 8,
  "fp": "lqee8fhe4ukzk1z6"
}
```

注意复制下来的时候，放到你的 IDE 里面时，在这个变量的前后要加上两个反引号。（就是 Tab 按键上面那个符号。因为这个变量实际是字符串，不是对象。用反引号可以保留字符串中隐含的换行符。这里这个变量实际是含有换行符的字符串，有点像 python 里用三个单引号括起来的東西。）

```
49
50 var data = `{
51     "sua": "Windows NT 10.0; Win64; x64",
52     "pp": {
53         "p2": "dogcao"
54     },
55     "extend": {
56         "wd": 0,
57         "l": 0,
58         "ls": 5,
59         "wk": 0,
60         "bu1": "0.1.6",
61         "bu2": -1,
62         "bu3": 49,
63         "bu4": 0,
64         "bu5": 0,
65         "bu6": 30,
66         "bu7": "",
67         "bu8": 0
68     },
69     "pf": "Win32",
70     "random": "Az1VETij69L_",
71     "v": "h5_file_v4.9.1",
72     "canvas": "5b7096a993d442aef38ceb90cce4b485",
73     "webglFp": "a3e19bf2c47e1ad07028cb74d487f910",
74     "ccn": 8,
75     "fp": "lqee8fhe4ukal3e8"
76 }
```

我们接着往下看日志，发现情况又变化了。还是 case 47 这里，倒数第 3 个函数之前是 function _\$F7(_\$Fe) 这个函数。现在变成了一个匿名函数 f(_\$o, _\$g)了。说明 s 这个堆栈发生了变化。这也是 JSVMP 很难去扣代码的原因。因为堆栈在随时的压入和取出，同一个控制流的地方，多次运行时的函数和内容是截然不同的。

```
,
=====case47=====
s[s.length - 3] ---> f (_$o, _$g) {
  _$o = _$o - (0x1a4a + 0x1 * -0x21c8 + 0x3e * 0x27);
  var _$X = _$u[_$o];
  if (a046522e.shhiey === undefined) {
    var _$L = function(_$J) {...
arguments ---> __collect envCollect=
```

再往下看，就可以看到结果出来了。似乎是把之前的明文变成了一个 474 位的数组，然后再编码成了字符串。这里的_map 明显是 Base64 编码用的字码表，但是_map1 似乎是一个自定义的魔改的编码表，推测很有可能是一个魔改 Base64 生成了最后的字符串。

```
=====case47=====
s[s.length - 2] ---> {stringify: f, parse: f} i
  ▶ parse: f (_$FG)
  ▶ stringify: f (_$FG)
  ▶ [[Prototype]]: Object

arguments --->
  ▼ init {words: Array(119), sigBytes: 474} i
    sigBytes: 474
    ▶ words: (119) [2064261152, 577992033, 574234658, 1466527332, 1870099232, 1314136113, 808333371, 542599534, 909:
    ▶ [[Prototype]]: Object

=====case47=====
s[s.length - 2] --->
  ▼ {_map1: 'rqponmlkjihgfedcbaZYXWVUTSRQPONMLKJIHGFEDCBA-_9876543210zyxwvuts', _map: 'ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/'}
    ▶ encode: f (_$FI)
    ▶ parse: f (_$FI)
    ▶ stringify: f (_$FI)
    _map: "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/"
    _map1: "rqponmlkjihgfedcbaZYXWVUTSRQPONMLKJIHGFEDCBA-_9876543210zyxwvuts"
    ▶ _reverseMap: (123) [空 x43, 62, 空 x3, 63, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 空 x3, 64, 空 x3, 0, 1, 2,
    ▶ [[Prototype]]: Object

arguments ---> of7ruCLj1DkQAC0R2aYSDS1dGWVP-ipjxjpPFipjLDrgzrJdJz1TIipjLDrgJrYfyT1ezboS3PITIKIf7PofHmVfG0oeIiISJGy
CUBjrJdJ31QHyVT5ipjLDrgJj4f9G1WJrJdJT1Pjrpjh7ZMLrJp7rJdJLY0JipjLrpjh7JjJrJdJPY0JipjLrpjh7pf4rJdJTY0JipjLrpjh7pfLDJ
3kjLDLj8m1TEu1SJRJdJjoPJrpjLrJpwqJdJrkPJrpjh7Jj3ToNL-oe1zVRUq5d7zp6rpWdq5P0u1S9G1WJrJdJnV04ipjLD7N
```

我们回到 IDE 中看一下变量的长度，刚好是 474，更加肯定了我们的猜测。

```
49
50 var data = `{
51   "sua": "Windows NT 10.0; Win64; x64",
52   "pp": {
53     "p2": "dogcao"
54   },
55   "extend": {
56     "wd": 0,
57     "l": 0,
58     "ls": 5,
59     "wk": 0,
60     "bu1": "0.1.6",
61     "bu2": -1,
62     "bu3": 49,
63     "bu4": 0,
64     "bu5": 0,
65     "bu6": 30,
66     "bu7": "",
67     "bu8": 0
68   },
69   "pf": "Win32",
70   "random": "AzlVETij69L_",
71   "v": "h5_file_v4.9.1",
72   "canvas": "5b7096a993d442aef38ceb90cce4b485",
73   "webglFp": "a3e19bf2c47e1ad07028cb74d487f910",
74   "ccn": 8,
75   "fp": "lqee8fhe4ukzklz6"
76 }`
77
78 console.log(data.length) // 474
79
```

另外观察函数名有 parse, encode, stringify, 看起来似乎都是和编码解码相关的函数名称。

此外我们可以在 IDE 中运行下面这个代码把字符串转成 sigBytes Words 格式。可以看到转换后的结果和网站上是一样的。（如果不一样说明你的反引号中间的部分没有和网站上保持一致，可能是你多加了空格或者 Tab 缩进。）

```
1  const CryptoJS = require("crypto-js");
2  var data = `{
3    "sua": "Windows NT 10.0; Win64; x64",
4    "pp": {
5      "p2": "dogcao"
6    },
7    "extend": {
8      "wd": 0,
9      "l": 0,
10     "ls": 5,
11     "wk": 0,
12     "bu1": "0.1.6",
13     "bu2": -1,
14     "bu3": 49,
15     "bu4": 0,
16     "bu5": 0,
17     "bu6": 30,
18     "bu7": "",
19     "bu8": 0
20   },
21   "pf": "Win32",
22   "random": "Az1VETij69L_",
23   "v": "h5_file_v4.9.1",
24   "canvas": "5b7096a993d442aef38ceb90cce4b485",
25   "webglFp": "a3e19bf2c47e1ad07028cb74d487f910",
26   "ccn": 8,
27   "fp": "lqee8fhe4ukzk1z6"
28 }`
29
30 // 生成 WordArray 对象
31 const keyWordArray = CryptoJS.enc.Utf8.parse(data);
32 console.log(keyWordArray);
33
34
35
```

```
{
  words: [
    2064261152, 577992033, 574234658, 1466527332, 1870099232, 1314136113,
    808333371, 542599534, 909392672, 2016818210, 738861088, 577794082,
    975207178, 538976288, 577778210, 975184484, 1869046625, 1864501792,
    545074186, 538976887, 2020894062, 1679964704, 2064261152, 538976887,
    1679964704, 808192544, 538976290, 1814182432, 808192544, 538976290,
    1819484730, 540355594, 538976288, 578251554, 975188012, 169877536,
    539124341, 824326688, 573582897, 775299628, 169877536, 539124341,
    841103904, 758197258, 538976288, 576877875, 574234676, 959187488,
    538976290, 1651848226, 975188012, 169877536, 539124341, 891435552,
    808192544, 538976290, 1651848738, 975188784, 738861088, 538976866,
    1966547514, 539107884, 169877536, 539124341, 941767200, 805969952,
    2100038176, 539127910, 574234658, 1466527283, 841100298, 538976882,
    1634624623, 1830959648, 574716524, 1447384169, 1781938508, 1596075018,
    538976886, 574234658, 1748328294, 1768711519, 1983131193, 774971948,
    169877538, 1667329654, 1634935354, 539112802, 925907254, 1631140147,
    1681142834, 1634035251, 946038114, 959472483, 1697931828, 943006252,
    169877538, 2003133031, 1816555554, 975184481,
    ... 19 more items
  ],
  sigBytes: 474
}
```

```
arguments ---> ▼ init {words: Array(119), sigBytes: 474} ⓘ
  sigBytes: 474
  words: Array(119)
    ▼ [0 .. 99]
      0: 2064261152
      1: 577992033
      2: 574234658
      3: 1466527332
      4: 1870099232
      5: 1314136113
      6: 808333371
      7: 542599534
      8: 909392672
      9: 2016818210
      10: 738861088
      11: 577794082
      12: 975207178
      13: 538976288
      14: 577778210
      15: 975184484
      16: 1869046625
      17: 1864501792
      18: 545074186
      19: 538976869
      20: 2020894062
      21: 1679964704
      22: 2064261152
      23: 538976887
      24: 1679964704
      25: 808192544
      26: 538976290
      27: 1814182432
      28: 808192544
      29: 538976290
```


第二章：详细分析

先讲一下 Base64 编码是怎么回事。Base64 编码就是把原始的文字三个一组的先转成 ASCII 码，然后再转成 8 位的二进制拼接起来（ $3 \times 8 = 24$ 位），最后再把二进制数字重新按 6 位进行分组，每 6 位为一组（ $4 \times 6 = 24$ 位），依照码表生成 Base64 编码后的字符串，生成 4 个字符串。

原文	c				a				t															
ASCII 编码	99				97				116															
二进制位	0	1	1	0	0	0	1	1	0	1	1	0	0	0	0	1	0	1	1	1	0	1	0	0
索引	24				54				5				52											
Base64	Y				2				F				0											

如果字原始的文字不能被三整除，就在后面补 0 和等号。

原文	c		
ASCII 编码	99		
二进制位	0 1 1 0 0 0 1 1	0 0 0 0	
索引	24	48	
Base64	Y	w	=

具体请参考：<https://zhuanlan.zhihu.com/p/111700349>

标准 Base64 索引表

数值	字符		数值	字符		数值	字符		数值	字符
0	A		16	Q		32	g		48	w
1	B		17	R		33	h		49	x
2	C		18	S		34	i		50	y
3	D		19	T		35	j		51	z
4	E		20	U		36	k		52	0
5	F		21	V		37	l		53	1
6	G		22	W		38	m		54	2
7	H		23	X		39	n		55	3
8	I		24	Y		40	o		56	4
9	J		25	Z		41	p		57	5
10	K		26	a		42	q		58	6
11	L		27	b		43	r		59	7
12	M		28	c		44	s		60	8
13	N		29	d		45	t		61	9
14	O		30	e		46	u		62	+
15	P		31	f		47	v		63	/

回到插桩，我们观察日志并且根据经验可以知道，sigBytes 一般会作为加密的最直接的输入，_map1 又很像是魔改 Base64 的编码表。所以最后这次 case 47 很有可能就是具体 Base64 编码的入口，于是我们在 Case 47 这里下一个断点（也可以下一个条件断点，自己写一个条件），当循环到第 5 次，就是出现_map 的时候跳入进去（注意别跑过了），这时候发现跳转到了 encode 函数中。

```
=====case47=====
s[s.length - 2] ---> ▼ {stringify: f, parse: f} ⓘ
  ▶ parse: f (/_$FG)
  ▶ stringify: f (/_$FG)
  ▶ [[Prototype]]: Object

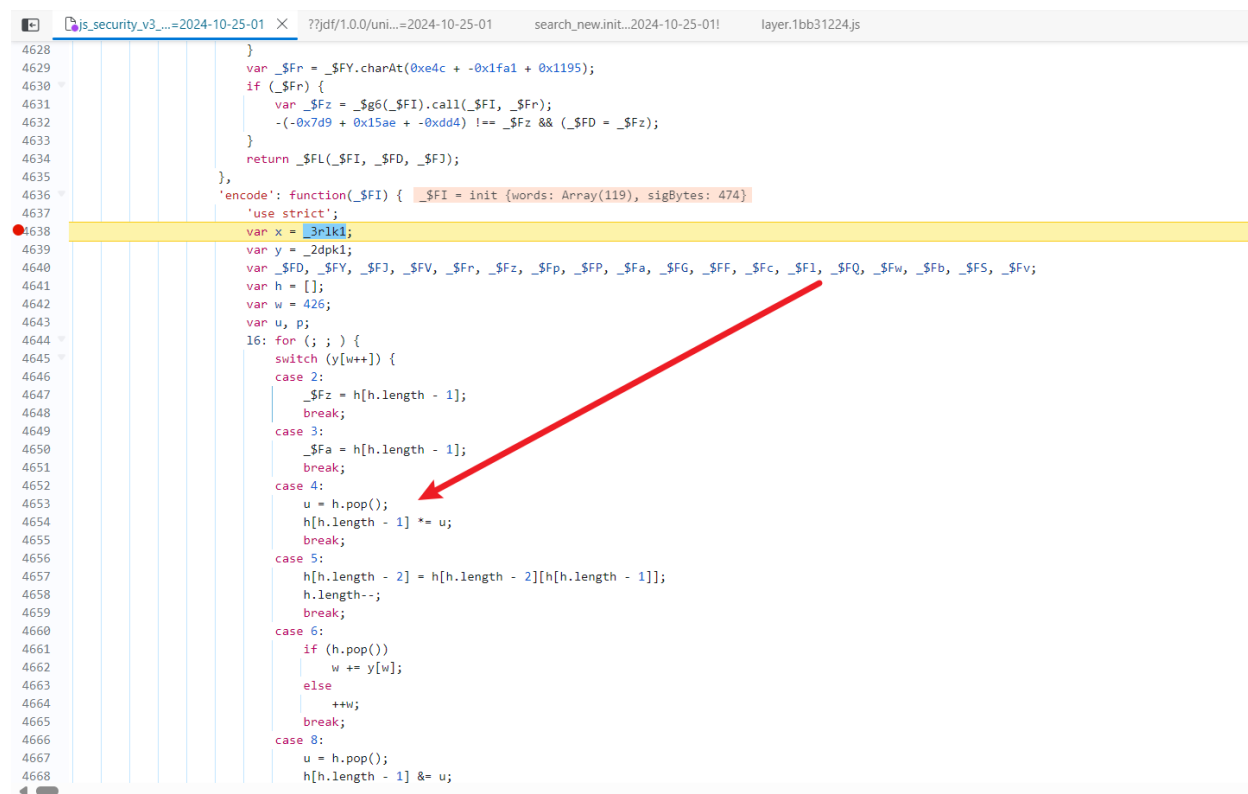
arguments --->
  ▼ init {words: Array(119), sigBytes: 474} ⓘ
    sigBytes: 474
    ▶ words: (119) [2064261152, 577992033, 574234658, 1466527332, 1870099232, 1314136113, 808333371, 542599534, 909:
    ▶ [[Prototype]]: Object

=====case47=====
s[s.length - 2] --->
  ▼ {_map1: 'rqponmlkjihgfedcbaZYXWVUTSRQPONMLKJIHGFEDCBA-_9876543210zyxwvuts', _map: 'ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/'}
    ▶ encode: f (/_$FI)
    ▶ parse: f (/_$FI)
    ▶ stringify: f (/_$FI)
    _map: "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/"
    _map1: "rqponmlkjihgfedcbaZYXWVUTSRQPONMLKJIHGFEDCBA-_9876543210zyxwvuts"
    ▶ _reverseMap: (123) [空 x43, 62, 空 x3, 63, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 空 x3, 64, 空 x3, 0, 1, 2,
    ▶ [[Prototype]]: Object

arguments ---> of7ruClj1DkQAC0R2aYSDSldGWVP-ipjxjpPFipjLDrgznJdJz1TIipjLDrgJrYfyT1ezboS3PITIKIf7PofHmVfG0oeIiISJGv
CUBJrJdJ31QHyVT5ipjLDrgJj4f9G1WJrJdJT1Pjrpjh7ZMLrJp7rJdJLY0jipjLrpjh7jJrJdJPY0jipjLrpjh7pf4rJdJTY0jipjLrpjh7pfLDJ
3kjLDLj8m1TEu1SjrJdJjoPjrpjLrJpwqJdJrkPjrpjh7jJ3ToNL-oe1zVRUq5d7zpf6rpWdq5P0u1S9G1WJrJdJnVO4ipjLD7N
```

```
0/400
8741     s.push(_$ru);
8742     break;
8743 case 46:
8744     s.push(_$GV);
8745     break;
8746 case 47:
8747     console.log("=====case47=====")
8748     if (s[s.length - 2] != null) {
8749         console.log("s[s.length - 2] --->", s[s.length - 2])
8750         console.log("arguments --->", h.call(s[s.length - 3], s[s.length - 2], s[s.length - 1]))
8751         s[s.length - 3] = h.call(s[s.length - 3], s[s.length - 2], s[s.length - 1]);
8752         s.length -= 2;
8753     } else {
8754         x = s[s.length - 3];
8755         console.log("s[s.length - 3] --->", s[s.length - 3])
8756         console.log("arguments --->", x(s[s.length - 1]))
8757         s[s.length - 3] = x(s[s.length - 1]);
8758         s.length -= 2;
8759     }
8760     break;
8761 case 56:
8762     s.push(_$aU);
8763     break;
8764 case 58:
8765     e += j[e];
8766     break;
8767 case 61:
```

我滴个乖乖，这个 encode 函数也是一个 JSVMP，可以看到我们这里重点关注的就是 h 这个变量，这里 h 就是这个 JSVMP 函数的堆栈。还是老样子，在 call 和 apply 的地方打上 log。并且在 encode 程序开始这里下一个断点。



```
4628 }
4629 var _$Fr = _$FY.charAt(0xe4c + -0x1fa1 + 0x1195);
4630 if (_$Fr) {
4631   var _$Fz = _$g6(_$FI).call(_$FI, _$Fr);
4632   -(-0x7d9 + 0x15ae + -0xdd4) != _$Fz && (_$FD = _$Fz);
4633 }
4634 return _$FL(_$FI, _$FD, _$FJ);
4635 },
4636 'encode': function(_$FI) { _$FI = init {words: Array(119), sigBytes: 474}
4637   'use strict';
4638   var x = 3r1k1;
4639   var y = _2dpk1;
4640   var _$FD, _$FY, _$FJ, _$FV, _$Fr, _$Fz, _$Fp, _$FP, _$Fa, _$FG, _$FF, _$Fc, _$Fl, _$FQ, _$Fw, _$Fb, _$FS, _$Fv;
4641   var h = [];
4642   var w = 426;
4643   var u, p;
4644   16: for (; ; ) {
4645     switch (y[w++]) {
4646       case 2:
4647         _$Fz = h[h.length - 1];
4648         break;
4649       case 3:
4650         _$Fa = h[h.length - 1];
4651         break;
4652       case 4:
4653         u = h.pop();
4654         h[h.length - 1] *= u;
4655         break;
4656       case 5:
4657         h[h.length - 2] = h[h.length - 2][h[h.length - 1]];
4658         h.length--;
4659         break;
4660       case 6:
4661         if (h.pop())
4662           w += y[w];
4663         else
4664           ++w;
4665         break;
4666       case 8:
4667         u = h.pop();
4668         h[h.length - 1] &= u;
```

Case 11 这里是一个条件分支，和之前的 Case 47 有点像。

```
case 11:
  console.log("=====encode,case 11=====")
  if (h[h.length - 1] != null) {
    console.log("h[h.length - 2], h[h.length - 1] --->", h[h.length - 2], h[h.length - 1])
    h[h.length - 2] = x.call(h[h.length - 2], h[h.length - 1]);
    console.log("result --->", h[h.length - 2])
  } else {
    u = h[h.length - 2];
    console.log("h[h.length - 2] --->", h[h.length - 2])
    h[h.length - 2] = u();
    console.log("result --->", h[h.length - 2])
  }
  h.length--;
  break;
```

Case 35 也是一个条件分支。

```
5
5
7
3
3
3
1
2
3
4
5
5
7
3
9
case 35:
  console.log("=====encode case 35=====")
  if (h[h.length - 2] != null) {
    console.log("arguments --->",h[h.length - 3], h[h.length - 2], h[h.length - 1])
    h[h.length - 3] = x.call(h[h.length - 3], h[h.length - 2], h[h.length - 1]);
    console.log("result --->",h[h.length - 3])
    h.length -= 2;
  } else {
    u = h[h.length - 3];
    console.log("h[h.length - 3] --->",h[h.length - 3])
    h[h.length - 3] = u(h[h.length - 1]);
    console.log("result --->",h[h.length - 3])
    h.length -= 2;
  }
  break;
```

Case 57 打上日志。

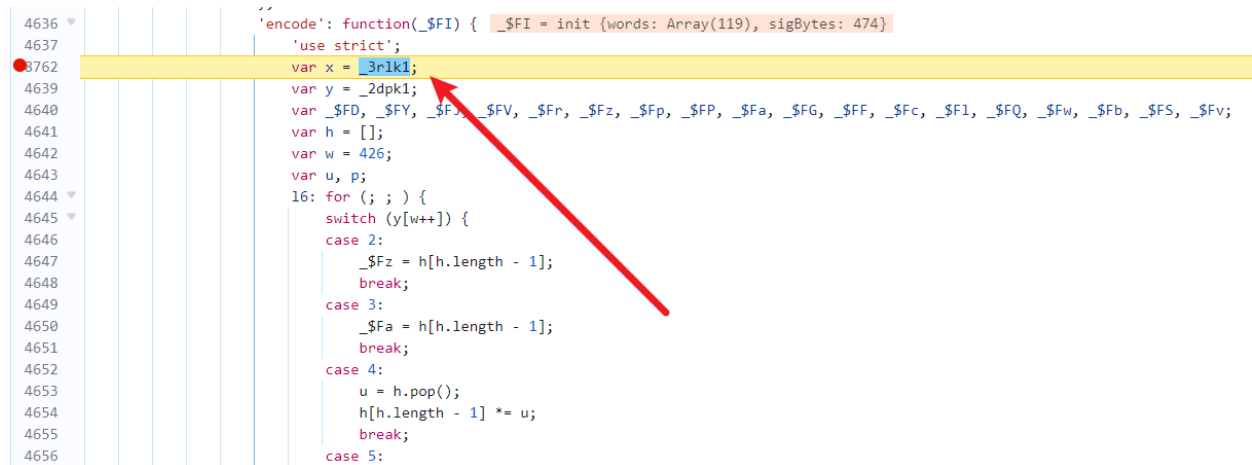
```
case 57:
  console.log("=====encode case 57=====")
  console.log("arguments --->",h[h.length - 2], h[h.length - 1])
  h[h.length - 5] = x.call(h[h.length - 5], h[h.length - 4], h[h.length - 3], h[h.length - 2], h[h.length - 1]);
  console.log("result --->",h[h.length - 5])
  h.length -= 4;
  break;
```

Case 69 打上日志。这里 x.call 有四个参数，一般情况 JSVMP 在执行 call 的时候通常传入的是（要执行的具体的函数，参数 1，参数 2，参数 3，.....）。这里刚好一个函数加三个参数，很符合 Base64 编码的规律，因为 Base64 编码就是传入三个字符串，输出四个字符串，这里后续要重点观察。

```
case 69:
  console.log("=====encode case 69=====")
  console.log("arguments --->", h[h.length - 1])
  h[h.length - 4] = x.call(h[h.length - 4], h[h.length - 3], h[h.length - 2], h[h.length - 1]);
  console.log("result --->",h[h.length - 4])
  h.length -= 3;
  break;
```

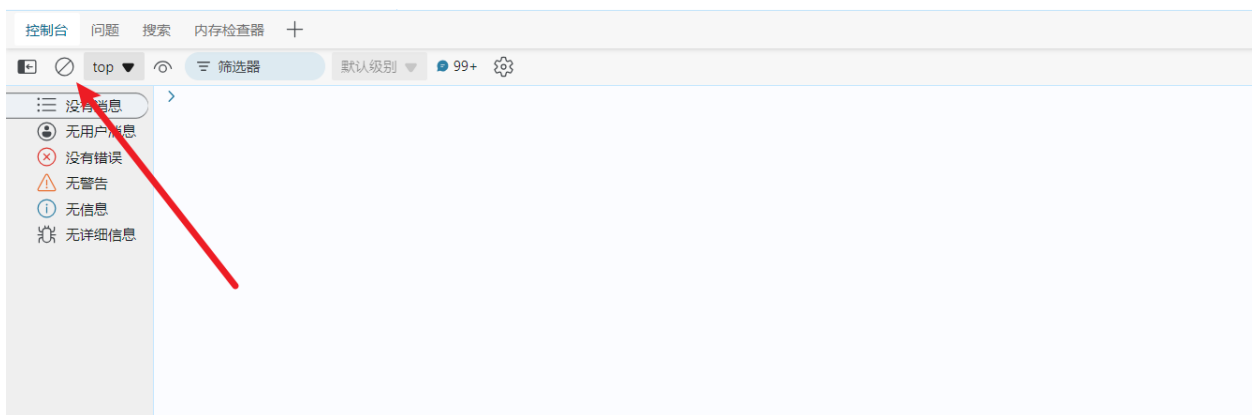
打完日志后刷新一下页面。（这次日志会很多，电脑配置不好的可能会卡或者很慢。）

当运行到 `encode` 函数这个断点的时候，点击控制台下面的清空按钮先清空一下日志，再继续运行到停止为止（很可能再次停到这里，因为一次网页访问可能会多次生成 H5ST）。



```
4636  'encode': function(_$FI) {  _$FI = init {words: Array(119), sigBytes: 474}
4637  'use strict';
4638  var x = 3r1k1;
4639  var y = _2dpk1;
4640  var _$FD, _$FY, _$FJ, _$FV, _$Fr, _$Fz, _$Fp, _$FP, _$Fa, _$FG, _$FF, _$Fc, _$F1, _$FQ, _$Fw, _$Fb, _$FS, _$Fv;
4641  var h = [];
4642  var w = 426;
4643  var u, p;
4644  16: for (; ; ) {
4645      switch (y[w++]) {
4646          case 2:
4647              _$Fz = h[h.length - 1];
4648              break;
4649          case 3:
4650              _$Fa = h[h.length - 1];
4651              break;
4652          case 4:
4653              u = h.pop();
4654              h[h.length - 1] *= u;
4655              break;
4656          case 5:
```

先清空旧的日志。



观察日志发现，case 35 一开始就把一个 474 位数组末尾添加了三个 3，变成了 477 位长度。这 474 位不就刚好是之前咱们的明文或者说 sigBytes 数组么？

```
=====encode case 35=====
arguments ---> f call() { [native code] } f slice() { [native code] }
  Uint8Array(474) [123, 10, 32, 32, 34, 115, 117, 97, 34, 58, 32, 34, 87, 105, 110, 100, 111, 119, 115, 32, 78, 84, 32, 49, 48, 46, 48, 59, 32, 87, 105, 110, 54, 52, 59, 32, 10, 32, 32, 125, 44, 10, 32, 32, 34, 101, 120, 116, 101, 110, 100, 34, 58, 32, 123, 10, 32, 32, 32, 32, 34, 119, 100, 34, 58, 32, ...]

result --->
  (474) [123, 10, 32, 32, 34, 115, 117, 97, 34, 58, 32, 34, 87, 105, 110, 100, 111, 119, 115, 32, 78, 84, 32, 49, 48, 46, 48, 59, 32, 87, 105, 110, 54, 52, 59, 32, 120, 54, 2, 125, 44, 10, 32, 32, 34, 101, 120, 116, 101, 110, 100, 34, 58, 32, 123, 10, 32, 32, 32, 32, 34, 119, 100, 34, 58, 32, ...]

=====encode case 69=====
arguments --->
  (474) [123, 10, 32, 32, 34, 115, 117, 97, 34, 58, 32, 34, 87, 105, 110, 100, 111, 119, 115, 32, 78, 84, 32, 49, 48, 46, 48, 59, 32, 87, 105, 110, 54, 52, 59, 32, 120, 54, 2, 125, 44, 10, 32, 32, 34, 101, 120, 116, 101, 110, 100, 34, 58, 32, 123, 10, 32, 32, 32, 32, 34, 119, 100, 34, 58, 32, ...]

result ---> 474

=====encode case 35=====
arguments ---> f push() { [native code] }
  (474) [123, 10, 32, 32, 34, 115, 117, 97, 34, 58, 32, 34, 87, 105, 110, 100, 111, 119, 115, 32, 78, 84, 32, 49, 48, 46, 48, 59, 32, 87, 105, 110, 54, 52, 59, 32, 120, 54, 2, 125, 44, 10, 32, 32, 34, 101, 120, 116, 101, 110, 100, 34, 58, 32, 123, 10, 32, 32, 32, 32, 34, 119, 100, 34, 58, 32, ...]
3

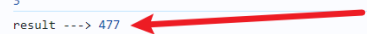
result ---> 475

=====encode case 35=====
arguments ---> f push() { [native code] }
  (475) [123, 10, 32, 32, 34, 115, 117, 97, 34, 58, 32, 34, 87, 105, 110, 100, 111, 119, 115, 32, 78, 84, 32, 49, 48, 46, 48, 59, 32, 87, 105, 110, 54, 52, 59, 32, 120, 54, 2, 125, 44, 10, 32, 32, 34, 101, 120, 116, 101, 110, 100, 34, 58, 32, 123, 10, 32, 32, 32, 32, 34, 119, 100, 34, 58, 32, ...]
3

result ---> 476

=====encode case 35=====
arguments ---> f push() { [native code] }
  (476) [123, 10, 32, 32, 34, 115, 117, 97, 34, 58, 32, 34, 87, 105, 110, 100, 111, 119, 115, 32, 78, 84, 32, 49, 48, 46, 48, 59, 32, 87, 105, 110, 54, 52, 59, 32, 120, 54, 2, 125, 44, 10, 32, 32, 34, 101, 120, 116, 101, 110, 100, 34, 58, 32, 123, 10, 32, 32, 32, 32, 34, 119, 100, 34, 58, 32, ...]
3

result ---> 477
```



我们可以用这段代码验证一下。把 474 位数组 copy 到 IDE 里面变成了一个对象。

```
1
2  var data = {
3      "0": 123,
4      "1": 10,
5      "2": 32,
6      "3": 32,
7      "4": 34,
8      "5": 115,
9      "6": 117,
10     "7": 97,
11     "8": 34,
12     "9": 58,
13     "10": 32,
```

用 Object.values()取出对象中的所有的值，再转换成 unitArray，最后 decode 成字符串。

```
1
2 > var data = { ...
477 }
478
479 unitArray = new Uint8Array(Object.values(data));
480
481 text = new TextDecoder().decode(unitArray);
482 console.log(text)
483
484
```

逆转回来的字符串与我们的预期相当的符合。

```
{
  "sua": "windows NT 10.0; Win64; x64",
  "pp": {
    "p2": "dogcao"
  },
  "extend": {
    "wd": 0,
    "l": 0,
    "ls": 5,
    "wk": 0,
    "bu1": "0.1.6",
    "bu2": -1,
    "bu3": 38,
    "bu4": 0,
    "bu5": 0,
    "bu6": 30,
    "bu7": "",
    "bu8": 0
  },
  "pf": "Win32",
  "random": "kLi0NqiBTBe6",
  "v": "h5_file_v4.9.1",
  "canvas": "5b7096a993d442aef38ceb90cce4b485",
  "webglFp": "a3e19bf2c47e1ad07028cb74d487f910",
  "ccn": 8,
  "fp": "lqee8fhe4ukzklz6"
}
```


接着往下看日志，似乎 Case 57 和 69 在三个一组的开始处理数组，怎么是先从 3,3,3 开始处理的，这 3,3,3 不是前面 Case 35 添加在 474 位数组最末尾的么，似乎是从末尾倒着三个三个的处理的。

```
=====encode case 57=====
arguments ---> 474 477
result ---> ▶ (3) [3, 3, 3]
=====encode case 69=====
arguments ---> ▶ (3) [3, 3, 3]
result ---> 3
=====encode case 35=====
h[h.length - 3] ---> f (_$F9) {
  var _$FO = _$F9.slice;
  return _$F9 === _$oq || _$oh(_$oq, _$F9) && _$FO === _$oq.slice ? _$oM : _$FO;
}
result ---> f slice() { [native code] }
=====encode case 57=====
arguments ---> 471 474
result ---> ▶ (3) [34, 10, 125]
=====encode case 69=====
arguments ---> ▶ (3) [34, 10, 125]
result ---> 6
=====encode case 35=====
h[h.length - 3] ---> f (_$F9) {
  var _$FO = _$F9.slice;
  return _$F9 === _$oq || _$oh(_$oq, _$F9) && _$FO === _$oq.slice ? _$oM : _$FO;
}
result ---> f slice() { [native code] }
=====encode case 57=====
arguments ---> 468 471
result ---> ▶ (3) [114, 101, 51]
=====encode case 69=====
arguments ---> ▶ (3) [114, 101, 51]
result ---> 9
=====encode case 35=====
```

```
452: 34
454: 34
455: 101
456: 53
457: 48
458: 118
459: 102
460: 113
461: 121
462: 108
463: 105
464: 101
465: 49
466: 105
467: 101
468: 114
469: 101
470: 51
471: 34
472: 10
473: 125
474: 3
475: 3
length: 477
▶ [[Prototype]]: Array(0)
3
```

接着往下翻日志，123,10,32 刚好也是最头的三位数字。说明是倒序处理完了 477 位数组。

```
result ---> ▶ (3) [117, 97, 34]
=====encode case 69=====
arguments ---> ▶ (3) [117, 97, 34]
result ---> 471
=====encode case 35=====
h[h.length - 3] ---> f (_, $F9) {
  var _$FO = _$F9.slice;
  return _$F9 === _$oq || _$oh(_$oq, _$F9) && _$FO === _$oq.slice ? _$oM : _$FO;
}
result ---> f slice() { [native code] }
=====encode case 57=====
arguments ---> 3 6
result ---> ▶ (3) [32, 34, 115]
=====encode case 69=====
arguments ---> ▶ (3) [32, 34, 115]
result ---> 474
=====encode case 35=====
h[h.length - 3] ---> f (_, $F9) {
  var _$FO = _$F9.slice;
  return _$F9 === _$oq || _$oh(_$oq, _$F9) && _$FO === _$oq.slice ? _$oM : _$FO;
}
result ---> f slice() { [native code] }
=====encode case 57=====
arguments ---> 0 3
result ---> ▶ (3) [123, 10, 32]
=====encode case 69=====
arguments ---> ▶ (3) [123, 10, 32]
result ---> 477
```

```
result --->
  UInt8Array(474) [123, 10, 32, 32, 34, 115, 117, 97, 34, 58, 32, 34, 10, 32, 32, 125, 44, 10, 32, 32, 34, 101, 120, 116, 101, 110, 100, 3, ...]
    ▼ [0 ... 99]
      0: 123
      1: 10
      2: 32
      3: 32
      4: 34
      5: 115
      6: 117
      7: 97
      8: 34
      9: 58
      10: 32
      11: 34
      12: 87
      13: 105
      14: 110
      15: 100
      16: 111
```

接着往下观察日志，在 477 位数字完成分组后（477/3=159 组），在 case 11 压入了一个新的函数，这个函数就是使用自定义的编码表进行 Base64 编码的具体的执行程序了。最后会生成 159*4=636 个字符，组合成魔改 Base64 编码后的字符串。按照 Base64 编码的理论，本来应当三个一组处理的，但是这里 JD 做了魔改后是一个一个处理的了，但是其实本质没变。

[rqponmlkjihgfedcbaZYXWVUTSRQPONMLKJIHGFEDCBA-_9876543210zyxwvuts]

```
=====encode case 69=====
arguments ---> ▶ (3) [123, 10, 32]
result ---> 477
=====encode case 35=====
arguments ---> f(_$f6) {
  'use strict';
  var b = _3rLk1;
  var h = _2dpk1;
  var _$FF, _$Fc;
  var p = [];
  var n = 129;
  - ▶ {tolwordArray: f, fromWordArray: f}
  ▶ (477) [3, 3, 3, 3, 34, 10, 125, 114, 101, 51, 49, 105, 101, 108, 105, 101, 102, 113, 121, 53, 48, 118, 32, 34, 101, 112, 34, 58, 32, 34, 102, 44, 10, 32, 58, 32, 56, 99, 110, 34, 3, 57, 98, 97, 51, 101, 58, 32, 34, 70, 112, 34, 98, 103, 108, 34, 119, 101, 10, 32, 32, 53, 34, 44, 98, 52, 56, 99, ...]
result ---> ▶ {$super: {...}, words: Array(120), sigBytes: 477, init: f}
=====encode, case 11=====
h[h.length - 2], h[h.length - 1] ---> f() {
  var _$FG = this.words
  , _$FF = this.sigBytes;
  _$FG[_$FF >> 0x1350 + -0x12a0 + -0xae] &= 0x1b3ac5437 + -0xd251f387 + 0x1 * 0x1ea59f4f << -0x25a1... ▶ {$super: {...}, words: Array(120), sigBytes: 477, init: f}
result ---> undefined
=====encode case 35=====
arguments ---> f(charAt()) { [native code] } rqponmlkjihgfedcbaZYXWVUTSRQPONMLKJIHGFEDCBA-_9876543210zyxwvuts 0
result ---> r
=====encode case 35=====
arguments ---> f(push()) { [native code] } ▶ [] r
result ---> 1
=====encode case 35=====
arguments ---> f(charAt()) { [native code] } rqponmlkjihgfedcbaZYXWVUTSRQPONMLKJIHGFEDCBA-_9876543210zyxwvuts 48
result ---> 7
=====encode case 35=====
arguments ---> f(push()) { [native code] } ▶ ['r'] 7
result ---> 2
=====encode case 35=====
arguments ---> f(charAt()) { [native code] } rqponmlkjihgfedcbaZYXWVUTSRQPONMLKJIHGFEDCBA-_9876543210zyxwvuts 12
result ---> f
=====encode case 35=====
arguments ---> f(push()) { [native code] } ▶ (2) ['r', '7'] f
result ---> 3
=====encode case 35=====
arguments ---> f(charAt()) { [native code] } rqponmlkjihgfedcbaZYXWVUTSRQPONMLKJIHGFEDCBA-_9876543210zyxwvuts 3
result ---> o
=====encode case 35=====
arguments ---> f(push()) { [native code] } ▶ (3) ['r', '7', 'f'] o
result ---> 4
=====encode case 35=====
arguments ---> f(charAt()) { [native code] } rqponmlkjihgfedcbaZYXWVUTSRQPONMLKJIHGFEDCBA-_9876543210zyxwvuts 8
result ---> j
=====encode case 35=====
arguments ---> f(push()) { [native code] } ▶ (4) ['r', '7', 'f', 'o'] j
result ---> 5
=====encode case 35=====
arguments ---> f(charAt()) { [native code] } rqponmlkjihgfedcbaZYXWVUTSRQPONMLKJIHGFEDCBA-_9876543210zyxwvuts 32
result ---> L
=====encode case 35=====
arguments ---> f(push()) { [native code] } ▶ (5) ['r', '7', 'f', 'o', 'j'] L
```

压入一个编码函数

取魔改编码表第0个

取出来r

取魔改编码表第48个

取出来7

取魔改编码表第12个

取出来f

取魔改编码表第3个

取出来o

依次压入一个数组

最后一共生成了 636 位的魔改 Base64 编码。

```
=====encode case 35=====
arguments ----> f push() { [native code] }
  ▶ (632) ['r', '7', 'f', 'o', 'j', 'l', 'c', 'u', 'p', 'f', 'x', '4', 'f', 'v', 'g', 'g', 'q', 'l', 'g', 'g', 's', 'e', 'm', 'y', 'e', 'v', 'q', 'i', 'j', 'p', 'l', 'g', 'p', 'p', 'j', 'x', 'j', 'p',
  'r', 'j', 'e', 'i', 't', 'y', 's', 'o', 'b', 'z', 't', 'i', 'p', '3', 'f', 'i', 'k', 'i', 'f', 'o', 'p', '7', 'f', 'v', 'm', 'h', 'e', 'o', 'o', 'g', 's', 'i', 'l', 'i', 'f', 'v', 'g', 'j', ...]
  N
result ----> 633

=====encode case 35=====
arguments ----> f charAt() { [native code] } rponmlkjihgfedcbaZYXWVUTSRQPONMLKJIHGFEDCBA_-9876543210zyxwvuts 48
result ----> 7

=====encode case 35=====
arguments ----> f push() { [native code] }
  ▶ (633) ['r', '7', 'f', 'o', 'j', 'l', 'c', 'u', 'p', 'f', 'x', '4', 'f', 'v', 'g', 'g', 'q', 'l', 'g', 'g', 's', 'e', 'm', 'y', 'e', 'v', 'q', 'i', 'j', 'p', 'l', 'g', 'p', 'p', 'j', 'x', 'j', 'p',
  'r', 'j', 'e', 'i', 't', 'y', 's', 'o', 'b', 'z', 't', 'i', 'p', '3', 'f', 'i', 'k', 'i', 'f', 'o', 'p', '7', 'f', 'v', 'm', 'h', 'e', 'o', 'o', 'g', 's', 'i', 'l', 'i', 'f', 'v', 'g', 'j', ...]
  7
result ----> 634

=====encode case 35=====
arguments ----> f charAt() { [native code] } rponmlkjihgfedcbaZYXWVUTSRQPONMLKJIHGFEDCBA_-9876543210zyxwvuts 40
result ----> 0

=====encode case 35=====
arguments ----> f push() { [native code] }
  ▶ (634) ['r', '7', 'f', 'o', 'j', 'l', 'c', 'u', 'p', 'f', 'x', '4', 'f', 'v', 'g', 'g', 'q', 'l', 'g', 'g', 's', 'e', 'm', 'y', 'e', 'v', 'q', 'i', 'j', 'p', 'l', 'g', 'p', 'p', 'j', 'x', 'j', 'p',
  'r', 'j', 'e', 'i', 't', 'y', 's', 'o', 'b', 'z', 't', 'i', 'p', '3', 'f', 'i', 'k', 'i', 'f', 'o', 'p', '7', 'f', 'v', 'm', 'h', 'e', 'o', 'o', 'g', 's', 'i', 'l', 'i', 'f', 'v', 'g', 'j', ...]
  0
result ----> 635

=====encode case 35=====
arguments ----> f charAt() { [native code] } rponmlkjihgfedcbaZYXWVUTSRQPONMLKJIHGFEDCBA_-9876543210zyxwvuts 32
result ----> L

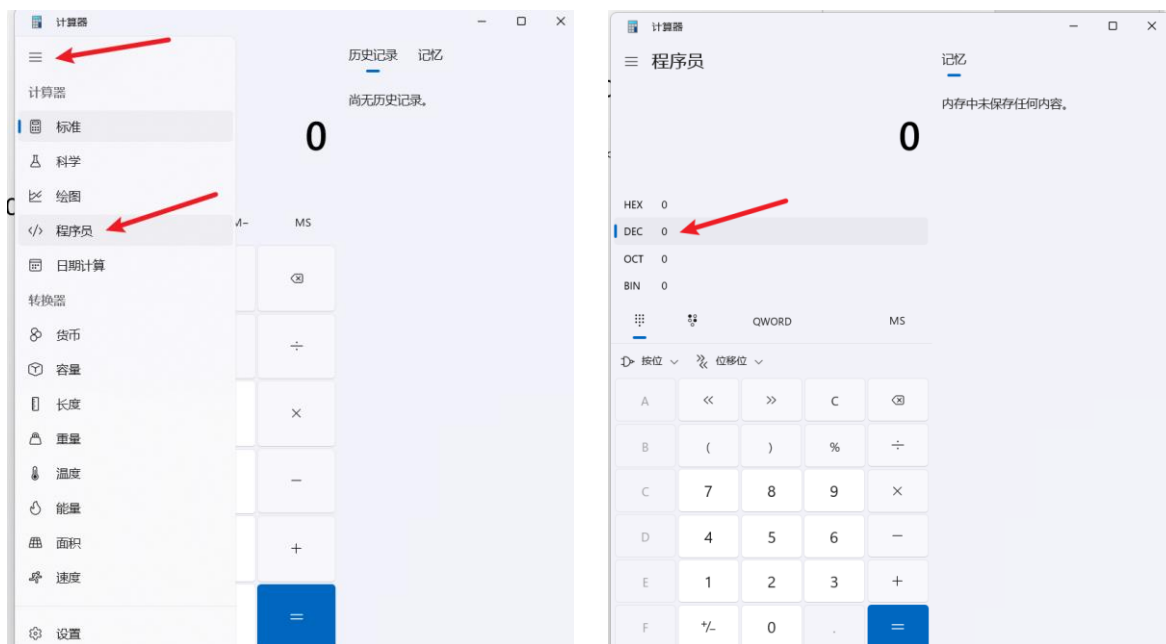
=====encode case 35=====
arguments ----> f push() { [native code] }
  ▶ (635) ['r', '7', 'f', 'o', 'j', 'l', 'c', 'u', 'p', 'f', 'x', '4', 'f', 'v', 'g', 'g', 'q', 'l', 'g', 'g', 's', 'e', 'm', 'y', 'e', 'v', 'q', 'i', 'j', 'p', 'l', 'g', 'p', 'p', 'j', 'x', 'j', 'p',
  'r', 'j', 'e', 'i', 't', 'y', 's', 'o', 'b', 'z', 't', 'i', 'p', '3', 'f', 'i', 'k', 'i', 'f', 'o', 'p', '7', 'f', 'v', 'm', 'h', 'e', 'o', 'o', 'g', 's', 'i', 'l', 'i', 'f', 'v', 'g', 'j', ...]
  0
  ▶ [0 - 99]
  ▶ [100 - 199]
  ▶ [200 - 299]
  ▶ [300 - 399]
  ▶ [400 - 499]
  ▶ [500 - 599]
  ▶ [600 - 634]
  length: 636
  ▶ [[Prototype]]: Array(0)
  L
result ----> 636
```



这里我们再验证一下生成的 r, 7, f, o....看看是否符合我们的预期。

由上面分析可知，是倒序三个一组进行编码的，第一组是[3,3,3], 第二组是[34,10,125] 等等。

我们用 windows 的计算器就可以完成验证。打开 windows 计算器的程序员模式。在 DEC 一栏输入十进制数字，可以再 BIN 一栏输出二进制数字。同理可以翻转换算。

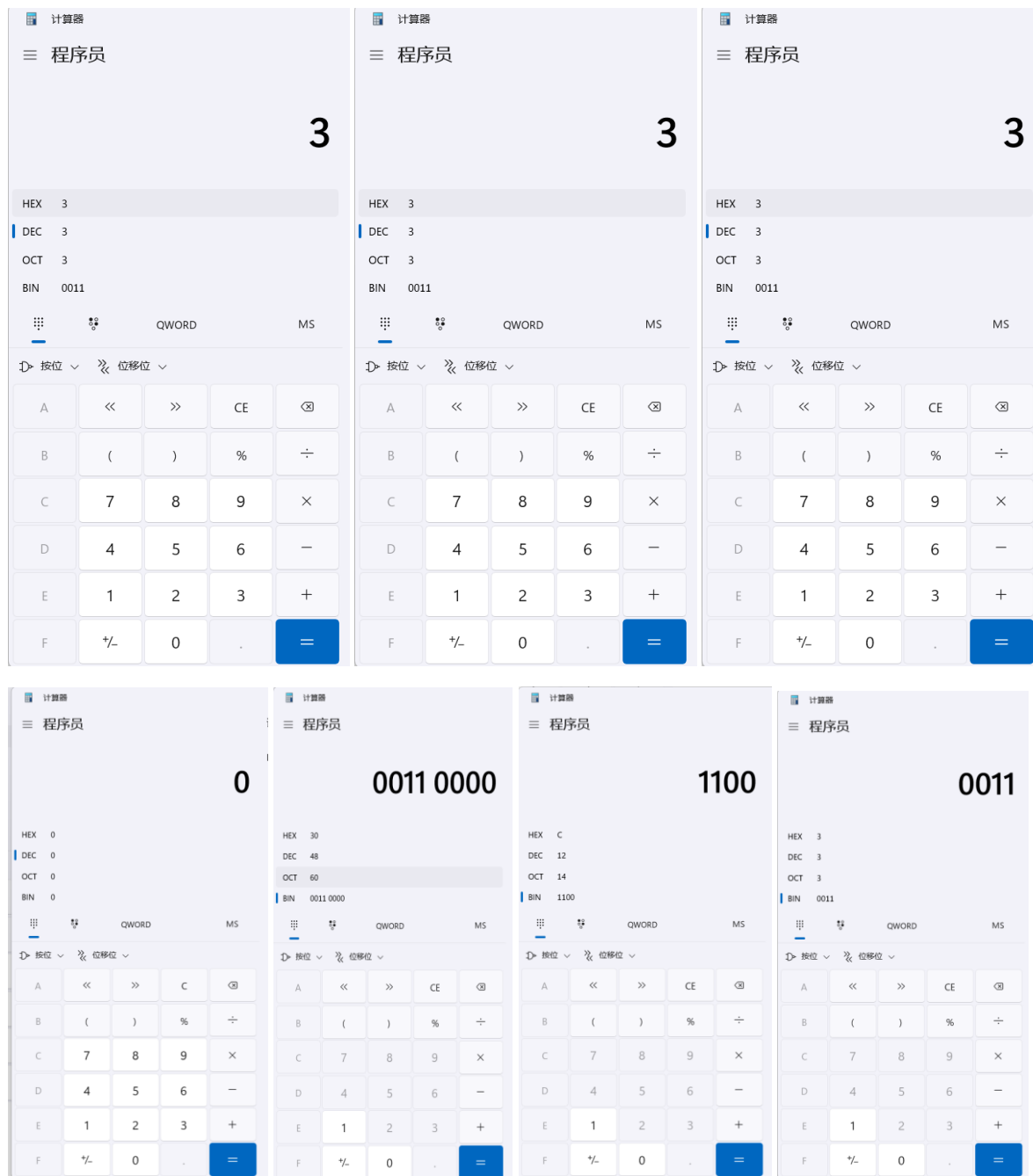


第一组的换算结果(计算器显示不足 8 位的时候, 前面要补 0 补足 8 位)

[3,3,3] ---转换二进制---> 00000011,00000011,00000011 ---拼接后按 6 位分组--->
000000,110000,001100,000011 ---转换回十进制--->[0,48,12,3]。

查魔改 Base64 编码表可得[r,7,f,o]

[rqponmlkjihgfedcbaZYXWVUTSRQPONMLKJIHGFEDCBA-_9876543210zyxwvuts] <-注意编号是 0 到 63

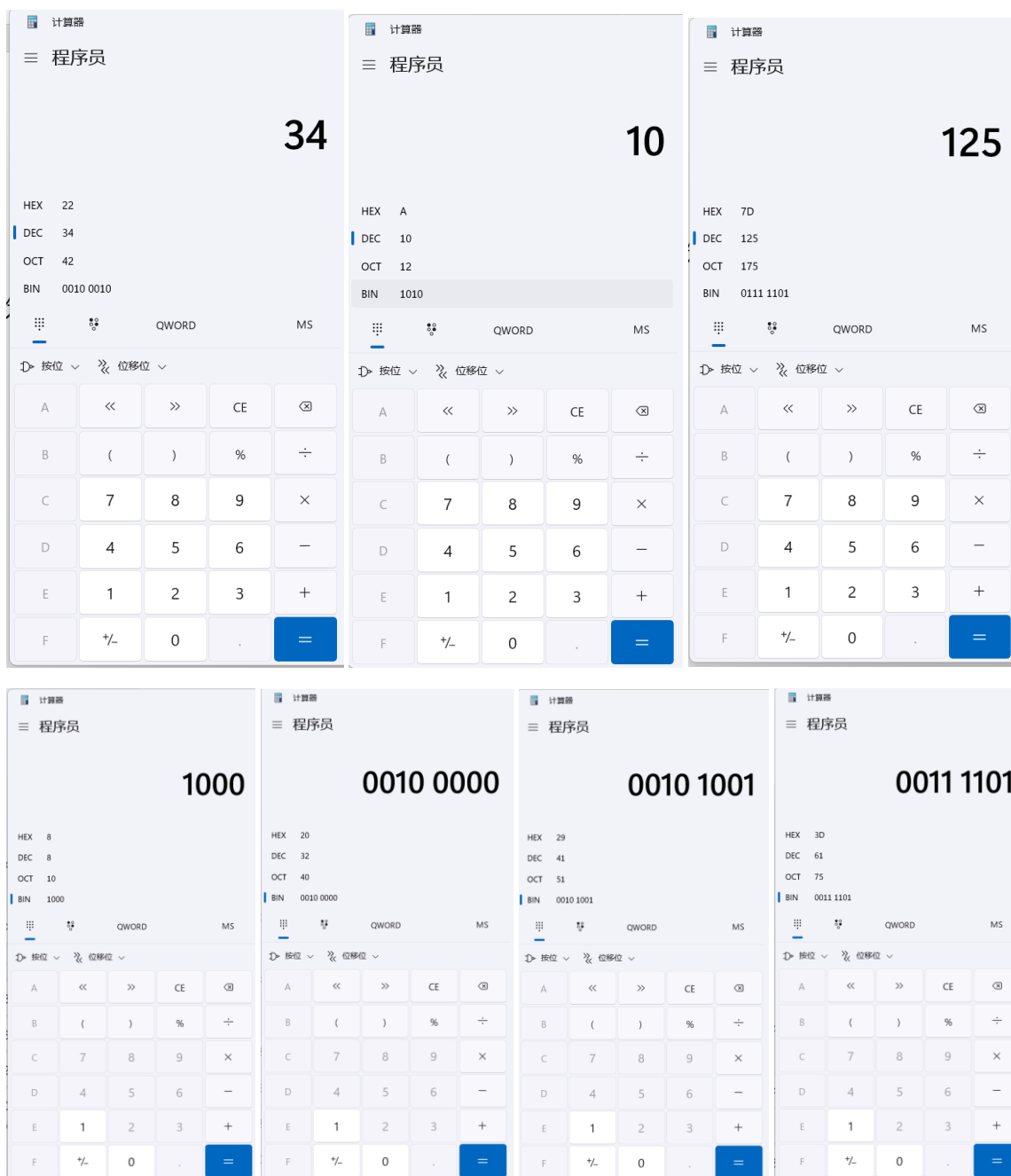


第二组的换算结果(计算器显示不足 8 位的时候, 前面要补 0 补足 8 位)

[34,10,125] ---转换二进制---> 00100010,00001010,01111101 ---拼接后按 6 位分组---> 001000,100000,101001, 111101---转换回十进制--->[8,32,41,61]。

查魔改 Base64 编码表可得[j,L,C,u]

[rqponmlkjihgfedcbaZYXWVUTSRQPONMLKJIHGFEDCBA-_9876543210zyxwvuts] <-注意编号是 0 到 63



再往下看日志，发现把生成的 636 个字符再次以每四个一组进行的反转。

```

    ...
    ▶ [[Prototype]]: Array(0)
L
result ---> 636
=====encode case 35=====
h[h.length - 3] ---> f (_$F9) {
    var _$FO = _$F9.slice;
    return _$F9 === _$oq || _$oh(_$oq, _$F9) && _$FO === _$oq.slice ? _$oM : _$FO;
}
result ---> f slice() { [native code] }
=====encode case 57=====
arguments ---> 0 4
result ---> ▶ (4) ['r', '7', 'f', 'o']
=====encode case 35=====
h[h.length - 3] ---> f (_$Fe) {
    var _$Fu = _$Fe.reverse;
    return _$Fe === _$ag || _$au(_$ag, _$Fe) && _$Fu === _$ag.reverse ? _$ao : _$Fu;
}
result ---> f reverse() { [native code] }
=====encode case 35=====
arguments ---> f call() { [native code] } f reverse() { [native code] } ▶ (4) ['r', '7', 'f', 'o']
result ---> ▶ (4) ['o', 'f', '7', 'r']
=====encode case 69=====
arguments ---> ▶ (4) ['o', 'f', '7', 'r']
result ---> 4
=====encode case 35=====
h[h.length - 3] ---> f (_$F9) {
    var _$FO = _$F9.slice;
    return _$F9 === _$oq || _$oh(_$oq, _$F9) && _$FO === _$oq.slice ? _$oM : _$FO;
}
result ---> f slice() { [native code] }
=====encode case 57=====
arguments ---> 4 8
result ---> ▶ (4) ['j', 'L', 'C', 'u']
=====encode case 35=====
h[h.length - 3] ---> f (_$Fe) {
    var _$Fu = _$Fe.reverse;
    return _$Fe === _$ag || _$au(_$ag, _$Fe) && _$Fu === _$ag.reverse ? _$ao : _$Fu;
}
result ---> f reverse() { [native code] }
=====encode case 35=====
arguments ---> f call() { [native code] } f reverse() { [native code] } ▶ (4) ['j', 'L', 'C', 'u']
result ---> ▶ (4) ['u', 'C', 'L', 'j']
=====encode case 69=====
arguments ---> ▶ (4) ['u', 'C', 'L', 'j']
result ---> 8
=====encode case 35=====
h[h.length - 3] ---> f (_$F9) {
    var _$FO = _$F9.slice;
    return _$F9 === _$oq || _$oh(_$oq, _$F9) && _$FO === _$oq.slice ? _$oM : _$FO;
}

```


反转完成后拼接起来就完成了第八段的运算了。至此日志分析完毕。

```
f
result ---> f slice() { [native code] }
=====encode case 57=====
arguments ---> 632 636
result ---> ► (4) ['N', '7', 'D', 'L']
=====encode case 35=====
h[h.length - 3] ---> f (_$Fe) {
  var _$Fu = _$Fe.reverse;
  return _$Fe === _$ag || _$au(_$ag, _$Fe) && _$Fu === _$ag.reverse ? _$ao : _$Fu;
}
result ---> f reverse() { [native code] }
=====encode case 35=====
arguments ---> f call() { [native code] } f reverse() { [native code] } ► (4) ['N', '7', 'D', 'L']
result ---> ► (4) ['L', 'D', '7', 'N']
=====encode case 69=====
arguments ---> ► (4) ['L', 'D', '7', 'N']
result ---> 636
=====encode case 35=====
arguments ---> f join() { [native code] }
  (636) ['o', 'f', '7', 'r', 'u', 'C', 'L', 'j', '4', 'X', 'F', 'P', 'G', 'V', 'f', 'G', 'G', 'L', 'Q', 'y', 'm', 'E', 'S', 'I', 'q', 'Y', 'e', 'G', 'i', 'p',
  ► 'Y', 'f', 'y', 'T', 'I', 'e', 'z', 'b', 'o', 'S', '3', 'P', 'I', 'T', 'I', 'K', 'I', 'f', '7', 'P', 'o', 'f', 'H', 'm', 'V', 'f', 'G', 'O', 'e', 'I', 'I', 'S', 'J', 'G', 'Y', 'f', 'G', 'e', 'Y', 'T', 'J', 'r', 'J', 'd', 'J', 'r', 'E', 'a', 'O', 'F', 'T', 'G', 'O', 'E', 'J', 'L', 'r', 'J', 'p', 'j', 'Z', 'e', 'z', 'b', 'I', 'T', '3', 'X', '1', 'I', 'q', 'Y', 'd', 'J', 'W', '1', 'T', 'z', 'f', 'I', 'S', 'G', 'm', 'F', 'j', 'Z', 'f', '9', 'H', 'I', 'g', '3', 'T', '0', 'U', 'G', '6', 'V', 'R', 'F', 'u', 'W', 'e', 'D', 'i', 'p', 'j', 'x', 'j', 'o', 'J', 'r', 'p', 'j', 'h', '7', 'J', 'j', '1', 'K', '3', 'Z', 'B', 'K', 'k', 'P', '4', 'r', '2', 'f', 'Z', 'W', 'G', 'R', 'J', 'r', 'J', 'd', 'J', '3', '1', 'Q', 'H', 'y', 'V', 'T', '5', 'i', 'p', 'j', 'L', 'D', 'r', 'g', 'J', '4', 'f', '9', 'G', '1', 'W', 'J', 'r', 'J', 'd', 'J', 'T', 'I', 'P', 'r', 'p', 'j', 'h', '7', 'Z', 'M', 'L', 'r', 'J', 'p', '7', 'r', 'J', 'd', 'J', 'L', 'Y', 'O', 'J', 'i', 'p', 'j', 'L', 'r', 'p', 'j', 'h', '7', 'J', 'j', 'r', 'J', 'd', 'J', 'P', 'Y', 'O', 'J', 'i', 'p', 'j', 'L', 'r', 'p', 'j', 'h', '7', 'Z', 'f', '4', 'r', 'J', 'd', 'J', 'T', 'Y', 'O', 'J', 'i', 'p', 'j', 'L', 'r', 'p', 'j', 'h', '7', 'p', 'f', 'L', 'D', 'I', 'j', 'r', 'o', 'j', 'x', 'j', 'p', 'S', '0', 'i', 'p', 'j', 'L', 'r', 'p', 'j', 'h', 'k', 'j', 'x', 'j', 'p', 'S', '9', 'W', '1', 'O', 'z', 'W', 'F', 'J', 'L', 'r', 'J', 'p', '3', 'k', 'j', 'L', 'D', 'L', 'j', '8', 'm', '1', 'T', 'e', 'u', '1', 'S', 'J', 'r', 'J', 'd', 'J', 'j', 'o', 'P', 'J', 'r', 'p', 'j', 'L', 'r', 'J', 'p', 'w', 'q', 'J', 'd', 'J', 'r', 'k', 'P', 'J', 'r', 'p', 'j', 'h', '7', 'J', 'j', '3', 'T', 'o', 'N', 'L', 'o', 'e', '1', 'z', 'V', 'R', 'U', 'q', '5', 'd', '7', 'z', 'p', 'f', '6', 'r', 'p', 'W', 'd', 'q', '5', 'P', '0', 'u', '1', 'S', '9', 'G', '1', 'W', 'J', 'r', 'J', 'd', 'J', 'n', 'V', 'O', '4', 'i', 'p', 'j', 'L', 'D', '7', 'N']
result ---> of7ruCLj4XFPGGVfGG1QymES1qYeG1pJxjpPF1pJLDrgzrJdJz1TIipJLDrgJrYfyT1ezboS3PITIKIf7PofHmVfG0oeIiISJGYfGeYTJrJdJrEa-OFTG0EjLrJp-jZeZbIT3X1TIqYdJW1TzfISGmFfjZf9HIg3T0UG6VRFuWeD1pJxjJ0Jrpjrh7Jj1K3ZBKKp4r2fZWGRJrJdJ31QHyt51pJLDrgJj4f9G1WJrJdJ1PJPrpjh7ZMLrJp7rJdJLYOJ1pJLrpjh7JjJrJdJPYOJ1pJLrpjh7Zf4rJdJTYOJ1pJLrpjh7pFLDIj2rojxjpS0ipjLrpjh-kjxjpS9W1OzWfJLrJp-3kjLDLj8m1TEu1S3rJdJjoPJrpjLrJpwqJdJrkPJrpjh7Jj3ToNL-oe1zVRUq5d7zp6rpWdq5P0u1S9G1WJrJdJnV04ipjLD7N
arguments ---> of7ruCLj4XFPGGVfGG1QymES1qYeG1pJxjpPF1pJLDrgzrJdJz1TIipJLDrgJrYfyT1ezboS3PITIKIf7PofHmVfG0oeIiISJGYfGeYTJrJdJrEa-OFTG0EjLrJp-jZeZbIT3X1TIqYdJW1TzfISGmFfjZf9HIg3T0UG6VRFuWeD1pJxjJ0Jrpjrh7Jj1K3ZBKKp4r2fZWGRJrJdJ31QHyt51pJLDrgJj4f9G1WJrJdJ1PJPrpjh7ZMLrJp7rJdJLYOJ1pJLrpjh7JjJrJdJPYOJ1pJLrpjh7Zf4rJdJTYOJ1pJLrpjh7pFLDIj2rojxjpS0ipjLrpjh-kjxjpS9W1OzWfJLrJp-3kjLDLj8m1TEu1S3rJdJjoPJrpjLrJpwqJdJrkPJrpjh7Jj3ToNL-oe1zVRUq5d7zp6rpWdq5P0u1S9G1WJrJdJnV04ipjLD7N
>
```

第三章，代码实现

在实现代码的时候，用 JS 或 Python 或者 C 等其他语言都是可以的。

找到我之前发的“加密算法合集”（没有可以私我）里的“Str_Base64_manual.js”文件。稍微改动一下即可。

把这里替换成 JD 魔改的 Base64 字码表。

```
// 2024-10-15 15:38:17, Str_Base64_manual.js, ...
// 不借助官方或者第三方库，纯源码实现

var Base64 = {
  _keyStr: "rqponmlkjihgfedcbaZYXWVUTSRQPONMLKJIHGFEDCBA-_9876543210zyxwvuts",

  encode: function (input) {
    var output = "";
    var chr1, chr2, chr3, enc1, enc2, enc3, enc4;
    var i = 0;
    input = Base64.utf8Encode(input);
    while (i < input.length) {
      chr1 = input.charCodeAt(i++);
      chr2 = input.charCodeAt(i++);
      chr3 = input.charCodeAt(i++);
      enc1 = chr1 >> 2;
      enc2 = ((chr1 & 3) << 4) | (chr2 >> 4);
      enc3 = ((chr2 & 15) << 2) | (chr3 >> 6);
      enc4 = chr3 & 63;
      if (isNaN(chr2)) {
        enc3 = enc4 = 64;
      } else if (isNaN(chr3)) {
        enc4 = 64;
      }
      output += Base64._keyStr.charAt(enc1) + Base64._keyStr.charAt(enc2) +
        Base64._keyStr.charAt(enc3) + Base64._keyStr.charAt(enc4);
    }
    return output;
  },
  utf8Encode: function (string) {
    string = string.replace(/\r\n/g, "\n");
    var utfText = "";
    for (var n = 0; n < string.length; n++) {
      var c = string.charCodeAt(n);
      if (c < 128) {
        utfText += String.fromCharCode(c);
      } else if ((c > 127) && (c < 2048)) {
        utfText += String.fromCharCode((c >> 6) | 192);
        utfText += String.fromCharCode((c & 63) | 128);
      } else {

```

生成的数据做一下反序处理，拼接一下[3,3,3]生成的四个字符串。

```
79
80
81 var resultEncoded = Base64.encode(data);
82 const reversedStr = [...resultEncoded].reverse().join('');
83 result = 'of7r' + reversedStr
84 console.log("第八组数据结果为--->", result);
85
86
```

最后验证结果与网页完全一致。

[illegible]

后记：

这里我补的是[3,3,3]，但是其实我不确定你的电脑环境里面会补什么，有可能例如补一个[1]或[3]以凑足 3 的整数倍长度。或者补两个[2,2]或两个[3,3]来凑足 3 的整数倍长度。这个需要你自己去验证了。

如果你第一天没有做完，第二天重启电脑后在 sign 断点进入时会讲到右边这个奇怪的位置去。

这是因为你的浏览器控制台里的“启用本地替代”功能被关闭了，并且 JD 重新刷新缓存。

search_newinit...2024-10-25-01 >>

```

159 var ojd = readCookie('tpuc-ojd') && ojd = '1-2010-0-0'
160 var area = (ddjd==null && ddjd.length>0)?ddjd.split('.')[0]:res
161 var colorParam = { colorParam: {appid: 'search-pc-java',
162 appid: 'search-pc-java',
163 functionalId: functionalId,    functionId = "pc_search_s_new"
164 client: 'pc',
165 clientVersion: '1.0.0',
166 t: new Date().getTime(),
167 body: JSON.stringify(paramJson) } paramJson = {keyword:
168 }
169
170 try{
171 var colorParamSign = JSON.parse(JSON.stringify(colorParam))
172 colorParamSign['body'] = SHA256(colorParam.body);
173 colorParamSign['loginType'] = '3';
174 colorParamSign['uid'] = jda;    sda = "143920055.173012603993711"
175 colorParamSign['area'] = area    area = "1_2010_0_0"
176 var that = this; that.$Window(window: Window, self: Window)
177 window.PSign.esign(colorParamSign).then(function(signed
178 window.h5stURI = encodeURI(signedParams.h5st)
179 colorParam['h5st'] = h5stURI;
180 //var url = getColorApiUrl(functionId,paramJson);
181 getJsToken(function(res){
182 if(res && res.jsToken){
183 colorParam['x-api-eid-token'] = res.jsToken;
184 }
185 $.ajax({
186 url: colorApi,
187 async: true,
188 data: colorParam || {},
189 //dataType: 'json',
190 xhrFields: {
191 withCredentials: true,
192 }

```

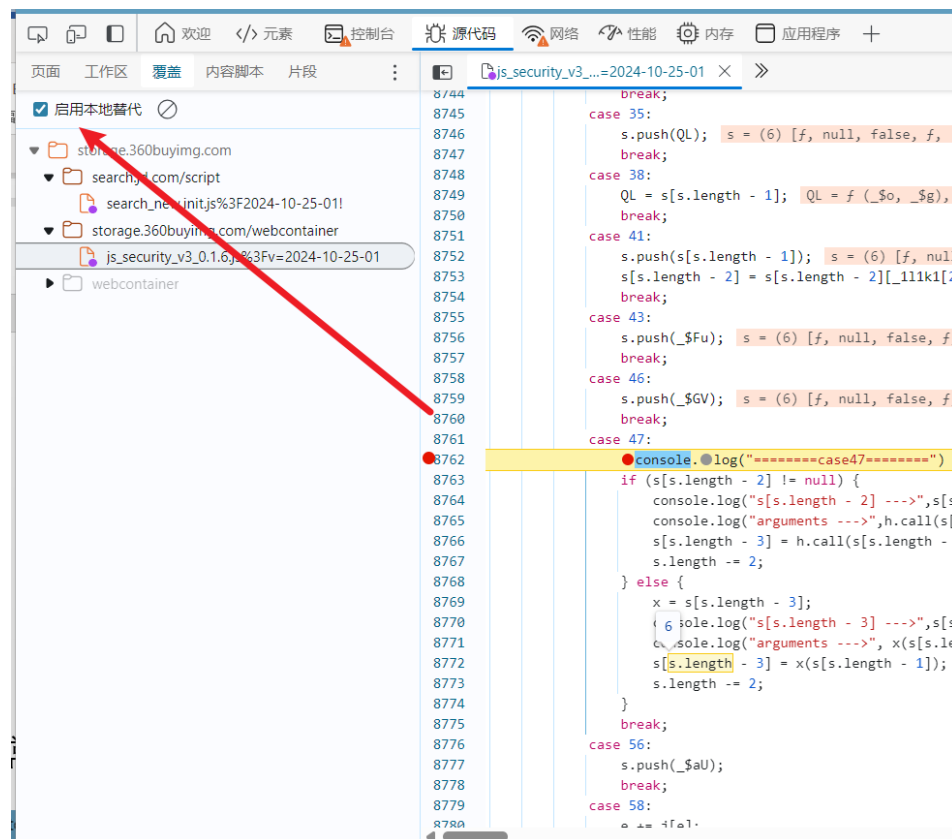
?jdfr/1.0.0/uni...=2024-10-25-01 >>

```

- if (!(!s && v === constructor) && z(r, v)) {
-   }
- }
- }
- return o
- });
- var kr = e.keys && function() {
-   return e.keys(arguments).length === 2
- }(1, 2);
- var Nr = e.keys && function() {
-   var r = e.keys(arguments);
-   return arguments.length !== 1 || r.length !== 1 || r[0] !==
- }(1);
- var Ur = e.keys;
- F(e, {
-   keys: function keys(r) { r = appid: 'search-pc-java', func
-     if (Nr(r)) {
-       return Ur(z(r))
-     } else {
-       return Ur(r)
-     }
-   }
- }, {kr || Nr;
- var Rr = -621987552e5;
- var Fr = "-000001";
- var Ar = Date.prototype.toISOString && new Date(Rr).toISOString()
- var Pr = Date.prototype.toISOString && new Date(-1).toISOString()
- F(Date.prototype, {
-   toISOString: function toISOString() {
-     var r, t, e, n, i;
-     if (!isFinite(this)) {
-       throw new RangeError("Date.prototype.toISOString cal

```

只要重新打开启用本地替代，然后再次刷新加载网页即可恢复正常了。



恭喜你完成了第一个 JSVMP 的逆向

